

Cómo resuelve EduFEM el sistema $Ku = F$

Matrices dispersas (COO, CSR, CSC), factorización LU, llenado, ordenamiento de mínimo grado sobre $K^\top + K$ y SuperLU, paso a paso

EduFEM — documento teórico del solucionador

23 de septiembre de 2026

Qué responde este documento

1. ¿EduFEM invierte la matriz de rigidez? No: la factoriza y resuelve por sustitución (§2).
2. ¿Qué es la factorización LU y por qué se factoriza? (§3).
3. ¿Qué es el formato COO, qué son CSR y CSC y cómo arma EduFEM la matriz? (§4).
4. ¿Qué es el *llenado* y por qué se reordenan las incógnitas? (§5).
5. ¿Qué hace el ordenamiento de mínimo grado sobre $K^\top + K$? (§6).
6. ¿Qué es SuperLU y qué hace por dentro? (§7).
7. ¿Cómo está todo esto en el código, qué hace NumPy y qué hace SciPy? (§8).
8. Preguntas que pueden aparecer en la defensa, con respuestas cortas (§9).

La respuesta corta

Sí: **EduFEM nunca calcula K^{-1}** . El sistema reducido $K_{ff}u_f = F_f$ se resuelve en dos tiempos:

1. SuperLU renumera las incógnitas con el ordenamiento de mínimo grado calculado sobre el grafo de $K^\top + K$ y factoriza la matriz renumerada, $P_r K_{ff} P_c = LU$, con L triangular inferior y U triangular superior;
2. con los dos factores obtiene u_f mediante una sustitución hacia adelante y otra hacia atrás. Invertir sería peor en todo. K^{-1} es una matriz llena (medido: el 100 % de sus coeficientes es no nulo), cuesta unas tres veces más operaciones que la LU incluso en formato denso y, con 33 024 GDL libres, ocuparía 8,7 GB; los factores L y U ocupan unos 65 MB. El reordenamiento es lo que mantiene chicos esos factores: con 8320 GDL, sin reordenar, $L + U$ tiene 52 485 872 coeficientes y la solución tarda 150 s; con el mínimo grado, 944 056 coeficientes y 0,13 s.

Cómo leer las cifras

Todo número medido de este documento sale de `generar_datos.py` (en esta misma carpeta), que llama al motor de EduFEM y a SciPy 1.17.1 / NumPy 2.4.6 (Python 3.13.9). El caso de prueba es la membrana de Cook con elementos Q9 —la misma del guion `tests/bench_timing.py` y de la tabla de tiempos de la tesis—, en mallas de $N \times N$ elementos. Los tiempos se midieron el 2026-09-23 en un Intel Core i5-3210M (2012) con 8 GB de RAM: varían con la carga del equipo, así que conviene leer las *razones* entre ellos más que los valores absolutos. Los ejemplos a mano se verificaron en aritmética exacta con el mismo guion.

Índice

1. El sistema que hay que resolver	3
1.1. De la malla a $\mathbf{Ku} = \mathbf{F}$	3
1.2. Restricciones y sistema reducido	3
1.3. Tres propiedades que el solucionador aprovecha	4
2. ¿Por qué no se invierte $\mathbf{K}$?	4
2.1. $\mathbf{u} = \mathbf{K}^{-1}\mathbf{F}$ es una notación, no un algoritmo	4
2.2. La inversa de una matriz dispersa es llena	5
2.3. Costo en operaciones	5
2.4. Memoria	5
2.5. Precisión	6
3. Factorización LU	6
3.1. La eliminación de Gauss, escrita como un producto	6
3.2. Ejemplo 1, paso a paso: la cadena de resortes	7
3.3. Sustitución hacia adelante y hacia atrás	8
3.4. ¿Por qué factorizar?	8
3.5. Pivoteo, y por qué una matriz SPD no lo necesita	8
3.6. Cholesky y $\mathbf{LDL}^\top$	8
4. Matrices dispersas: COO, CSR y CSC	9
4.1. Guardar solo lo que no es cero	9
4.2. COO: la lista de tripletes	9
4.3. CSR: filas comprimidas	10
4.4. CSC: columnas comprimidas	10
4.5. Qué formato para qué	10
4.6. El ensamblaje de EduFEM, línea por línea	10
5. El llenado: por qué importa el orden de las incógnitas	11
5.1. Qué es el llenado	11
5.2. Ejemplo 2: una estrella	11
5.3. Renumerar es permutar	12
5.4. El grafo de la matriz	13
5.5. ¿Cuál es el mejor orden?	13
6. Ordenamiento de mínimo grado sobre $\mathbf{K}^\top + \mathbf{K}$	13
6.1. La regla	13
6.2. Ejemplo 3, paso a paso: una malla de 2×2 elementos Q4	14
6.3. De nodos a GDL	15
6.4. Mínimo grado múltiple (MMD), el que usa SuperLU	15
6.5. ¿Por qué sobre $\mathbf{K}^\top + \mathbf{K}$?	15
6.6. Lo que se mide en EduFEM	16
7. SuperLU por dentro	19
7.1. Qué es	19
7.2. La factorización que calcula	19
7.3. Las cuatro fases	19
7.4. El árbol de eliminación	19
7.5. Factorización por columnas, «mirando a la izquierda»	20
7.6. Supernodos	20
7.7. Pivoteo por umbral	20

7.8. <code>spsolve</code> frente a <code>splu</code>	20
8. El solucionador de EduFEM, pieza por pieza	21
8.1. El recorrido completo	21
8.2. El código	21
8.3. Qué hace NumPy y qué hace SciPy	23
8.4. Traza de una resolución real	23
8.5. Cómo se comprueba que el solucionador está bien	23
9. Preguntas que pueden aparecer en la defensa	24
A. Conteo de operaciones	25
B. Cómo reproducir las cifras	26

1. El sistema que hay que resolver

1.1. De la malla a $Ku = F$

Cada nodo de la malla tiene dos grados de libertad (GDL): los desplazamientos u_x y u_y . Si la malla tiene N_{nodos} nodos, el vector de desplazamientos $\mathbf{u}$ tiene $2N_{\text{nodos}}$ componentes. EduFEM numera los GDL a partir del índice ordinal de cada nodo, $i = 0, 1, \dots$ (su posición en la lista ordenada de identificadores, `project.node_index_map`): el nodo de índice i ocupa las posiciones $2i$ (u_x) y $2i + 1$ (u_y). La matriz de rigidez global es la suma de las rigideces elementales,

$$\mathbf{K} = \sum_e \mathbf{A}_e^\top \mathbf{k}_e \mathbf{A}_e, \quad \mathbf{k}_e = \int_{\Omega_e} \mathbf{B}^\top \mathbf{D} \mathbf{B} t \, dA \approx \sum_g \mathbf{B}_g^\top \mathbf{D} \mathbf{B}_g t |J_g| w_g, \quad (1)$$

donde $\mathbf{A}_e$ es la matriz booleana que lleva los GDL locales del elemento a sus posiciones globales [23]. Nadie construye $\mathbf{A}_e$: en la práctica, *ensamblar* es sumar cada coeficiente $(\mathbf{k}_e)_{ab}$ en la posición global (i, j) que corresponde a los GDL locales a y b . La sección 4 muestra que eso es, literalmente, lo que hace el formato COO.

A lo largo del documento, $\text{nnz}(\mathbf{M})$ es el número de coeficientes no nulos (*number of nonzeros*) de una matriz $\mathbf{M}$, y n el número de incógnitas del sistema que se resuelve.

1.2. Restricciones y sistema reducido

Separando los GDL libres (f) de los restringidos (r), el equilibrio se escribe por bloques:

$$\begin{bmatrix} \mathbf{K}_{ff} & \mathbf{K}_{fr} \\ \mathbf{K}_{rf} & \mathbf{K}_{rr} \end{bmatrix} \begin{bmatrix} \mathbf{u}_f \\ \mathbf{u}_r \end{bmatrix} = \begin{bmatrix} \mathbf{F}_f \\ \mathbf{F}_r + \mathbf{R}_r \end{bmatrix}, \quad (2)$$

con $\mathbf{u}_r$ conocido (nulo en un apoyo fijo) y las reacciones $\mathbf{R}_r$ desconocidas. La primera fila de bloques es el **sistema reducido**, lo único que hay que resolver:

$$\mathbf{K}_{ff} \mathbf{u}_f = \mathbf{F}_f - \mathbf{K}_{fr} \mathbf{u}_r. \quad (3)$$

La segunda fila da las reacciones una vez conocido $\mathbf{u}$:

$$\mathbf{R}_r = \mathbf{K}_{rf} \mathbf{u}_f + \mathbf{K}_{rr} \mathbf{u}_r - \mathbf{F}_r. \quad (4)$$

En el código, `apply_boundary_conditions` extrae $\mathbf{K}_{ff}$ y $\mathbf{F}_f$ con una máscara booleana de GDL libres y resta $\mathbf{K}_{fr} \mathbf{u}_r$ solo si algún desplazamiento prescrito no es nulo (§8).

1.3. Tres propiedades que el solucionador aprovecha

(a) K_{ff} es simétrica. $k_e^\top = k_e$ porque $(B^\top DB)^\top = B^\top D^\top B$ y D es simétrica; una suma de matrices simétricas es simétrica, y K_{ff} es un bloque diagonal de K .

(b) K_{ff} es definida positiva. Para cualquier campo de desplazamientos v ,

$$v^\top K v = \int_{\Omega} \varepsilon(v)^\top D \varepsilon(v) t dA \geq 0, \quad (5)$$

que es el doble de la energía de deformación. Vale cero solo si $\varepsilon = \mathbf{0}$, es decir, en los movimientos de cuerpo rígido (dos traslaciones y una rotación en el plano): por eso K es singular. Al restringir suficientes GDL esos movimientos desaparecen y $v_f^\top K_{ff} v_f > 0$ para todo $v_f \neq \mathbf{0}$. Una matriz simétrica y definida positiva se abrevia SPD. Consecuencias: el sistema (3) tiene solución única, la eliminación de Gauss no necesita pivoteo (§3.5) y existe la factorización de Cholesky (§3.6).

(c) K es dispersa. $K_{ij} \neq 0$ solo si los GDL i y j pertenecen a nodos que comparten al menos un elemento (figura 1). Un nodo interior de una malla estructurada de Q4 toca 9 nodos —él mismo y sus 8 vecinos—, de modo que su fila tiene 18 coeficientes no nulos; en Q9 un nodo de esquina de elemento toca hasta 25 nodos, es decir, 50 no nulos (el máximo medido en la membrana de Cook es 50). Lo decisivo es que *el número de no nulos por fila no crece con la malla*, mientras que la longitud de la fila sí: con 544 GDL libres, K_{ff} tiene el 5,27 % de sus coeficientes no nulos; con 33 024, el 0,10 %.

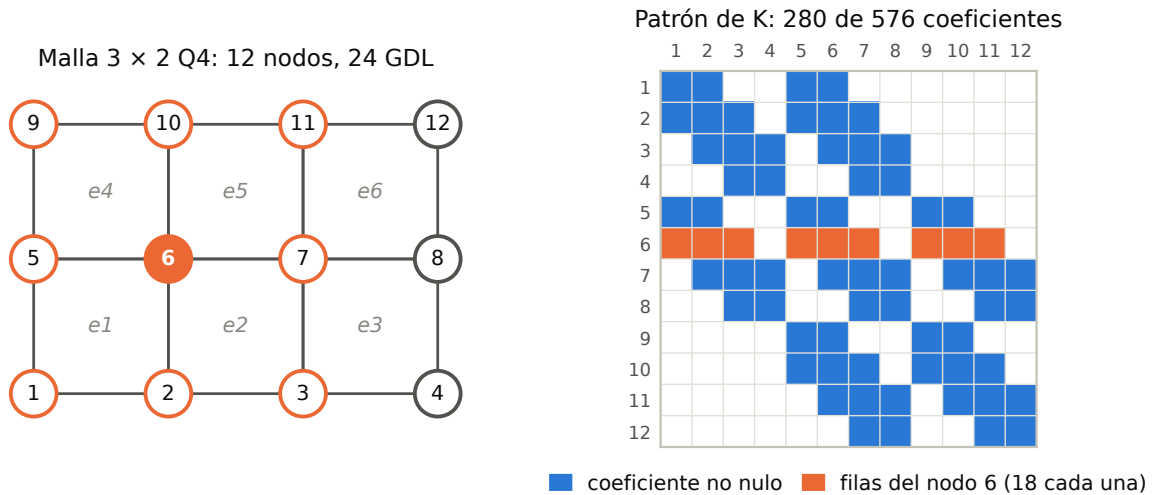


Figura 1: Por qué K es dispersa. Izquierda: malla de 3 × 2 elementos Q4; el nodo 6 comparte elemento con los ocho nodos de borde naranja. Derecha: patrón de K (24 × 24, un bloque de 2 × 2 por cada par de nodos): el bloque (i, j) es no nulo solo si los nodos i y j comparten un elemento. Las dos filas del nodo 6 tienen 18 coeficientes no nulos de 24; en una malla de miles de nodos seguirían teniendo 18. Se guardan 280 coeficientes; algunos se anulan o casi porque los aportes de elementos vecinos se cancelan por simetría (14 dan cero exacto en punto flotante), y se guardan igual: el formato disperso sigue el *patrón* que dicta la conectividad, no los valores.

2. ¿Por qué no se invierte K ?

2.1. $u = K^{-1}F$ es una notación, no un algoritmo

La expresión $u_f = K_{ff}^{-1}F_f$ afirma que la solución existe y es única; no dice cómo calcularla. Con un escalar pasa lo mismo: para resolver $7x = 21$ nadie calcula $7^{-1} = 0,142857\dots$ y después lo multiplica por 21 —con seis decimales daría 2,999997—; se divide y sale 3. Con matrices, «dividir» es *factorizar y sustituir*.

2.2. La inversa de una matriz dispersa es llena

La inversa de la rigidez tiene significado físico: es la matriz de **flexibilidad**. Su columna j resuelve $\mathbf{K}\mathbf{c}_j = \mathbf{e}_j$, así que contiene los desplazamientos de todos los GDL cuando actúa una fuerza unitaria en el GDL j . En un cuerpo conectado, una fuerza aplicada en un punto desplaza, en general, todos los puntos. Por eso $\mathbf{K}^{-1}$ es llena aunque $\mathbf{K}$ sea casi toda ceros.

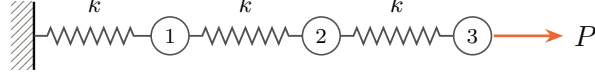


Figura 2: Ejemplo 1: tres resortes iguales en serie, empotrados a la izquierda. Cada nodo tiene un GDL (el desplazamiento horizontal).

Ejemplo 1. Para la cadena de la figura 2,

$$\mathbf{K} = k \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}, \quad \mathbf{K}^{-1} = \frac{1}{k} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \quad (\mathbf{K}^{-1})_{ij} = \frac{\text{mín}(i, j)}{k}. \quad (6)$$

La columna 3 de $\mathbf{K}^{-1}$ dice que una fuerza unitaria en el nodo 3 desplaza los nodos 1, 2 y 3 en $1/k$, $2/k$ y $3/k$: cada resorte transmite la fuerza y se alarga $1/k$. $\mathbf{K}$ tiene dos ceros —los nodos 1 y 3 no están unidos por un resorte— y $\mathbf{K}^{-1}$ no tiene ninguno, porque una fuerza en el nodo 3 sí mueve al nodo 1. En cambio, los factores $\mathbf{L}$ y $\mathbf{U}$ de esta misma matriz (§3.2) conservan los dos ceros. *Factorizar preserva la dispersión; invertir la destruye.*

Medido con EduFEM en la membrana de Cook: con 4704 GDL libres, $\mathbf{K}_{ff}$ tiene el 0,66 % de sus coeficientes no nulos y su inversa, el 100 % (0 ceros exactos).

2.3. Costo en operaciones

La tabla 1 cuenta las operaciones de punto flotante (*flops*) de cada camino para una matriz densa de orden n ; la deducción está en el apéndice A.

Tabla 1: Operaciones para resolver $\mathbf{K}\mathbf{u} = \mathbf{F}$ con una matriz densa de orden n .

Paso	Por factorización	Por inversa
Factorizar $\mathbf{K} = \mathbf{LU}$	$\frac{2}{3}n^3$	$\frac{2}{3}n^3$
Completar la inversa (n sistemas triangulares)	—	$\frac{4}{3}n^3$
Sustituir ($\mathbf{L}\mathbf{y} = \mathbf{F}$, $\mathbf{U}\mathbf{u} = \mathbf{y}$) o multiplicar ($\mathbf{K}^{-1}\mathbf{F}$)	$2n^2$	$2n^2$
Total	$\approx \frac{2}{3}n^3$	$\approx 2n^3$

Aun en formato denso, invertir cuesta tres veces más que factorizar. En formato disperso la brecha es mucho mayor. Con el mejor ordenamiento conocido para mallas planas, la disección anidada, la LU cuesta del orden de $n^{3/2}$ operaciones [11]; la inversa, con sus n^2 coeficientes, cuesta del orden de n^3 . El mínimo grado que usa EduFEM no tiene la garantía de la disección anidada, y en estas mallas sus operaciones crecen algo más rápido: contadas sobre los factores que produce SuperLU (tabla 4), como $n^{1.77}$. Sigue estando muy lejos del n^3 . Con 33 024 GDL la factorización hace $1,1 \times 10^9$ operaciones, contra $2,4 \times 10^{13}$ de una LU densa —unas 21 000 veces más— y $7,2 \times 10^{13}$ de la inversa.

2.4. Memoria

Guardar $\mathbf{K}^{-1}$ exige $8n^2$ bytes (8 por número en doble precisión). Con 33 024 GDL son 8,7 GB, más que toda la memoria del equipo en que se midieron estas cifras. Los factores $\mathbf{L}$ y $\mathbf{U}$ ordenados con mínimo grado tienen 5 386 440 coeficientes y ocupan unos 65 MB (unos 12 bytes por coeficiente: 8 del valor y 4 del índice).

2.5. Precisión

Hay también un argumento de exactitud. La solución por LU y sustitución es *regresivamente estable*: el $\hat{\mathbf{u}}$ calculado es la solución exacta de un sistema vecino $(\mathbf{K} + \Delta\mathbf{K})\hat{\mathbf{u}} = \mathbf{F}$ con $\|\Delta\mathbf{K}\|$ del orden del redondeo, $\varepsilon\|\mathbf{K}\|$ ($\varepsilon \approx 1,1 \times 10^{-16}$). Multiplicar por una inversa calculada no tiene esa garantía: su residuo $\mathbf{F} - \mathbf{K}\hat{\mathbf{u}}$ puede ser hasta $\kappa(\mathbf{K})$ veces mayor [3]. En la membrana de Cook la matriz está bien condicionada — $\kappa_2(\mathbf{K}_{ff}) \approx 2,3 \times 10^5$ con 2112 GDL— y los tres caminos de la tabla 2 dan residuos relativos del orden de 10^{-12} : aquí lo decisivo es el costo y la memoria. La precisión pasa a importar con matrices mal condicionadas: elementos muy distorsionados, coeficiente de Poisson cercano a 0,5 o materiales de rigideces muy dispares.

Tabla 2: Tres maneras de resolver el mismo $\mathbf{K}_{ff}\mathbf{u}_f = \mathbf{F}_f$ (membrana de Cook, Q9). *SuperLU*: `spsolve` con `MMD_AT_PLUS_A`, como EduFEM. *LU densa*: `numpy.linalg.solve`, que también factoriza (LAPACK), pero sobre los n^2 coeficientes. *Inversa*: `numpy.linalg.inv(K)` @ `F`. Residuo: $\|\mathbf{K}_{ff}\hat{\mathbf{u}}_f - \mathbf{F}_f\|_2 / \|\mathbf{F}_f\|_2$. Tiempos: mejor de 15 corridas (SuperLU) y de 7, 4 y 2 (densos, de menor a mayor n).

n	Tiempo (s)			Residuo relativo			No nulos de $\mathbf{K}_{ff}^{-1}$
	SuperLU	LU densa	Inversa	SuperLU	LU densa	Inversa	
544	0,0020	0,0092	0,027	$1,1 \times 10^{-12}$	$1,6 \times 10^{-12}$	$1,3 \times 10^{-12}$	100 %
2112	0,013	0,85	1,7	$3,0 \times 10^{-12}$	$7,5 \times 10^{-12}$	$3,9 \times 10^{-12}$	100 %
4704	0,027	3,1	13	$5,8 \times 10^{-12}$	$1,5 \times 10^{-11}$	$7,7 \times 10^{-12}$	100 %

Con 4704 GDL, la inversa tarda 13 s y SuperLU 0,027 s: 465 veces más. Incluso frente a la LU densa de NumPy, la inversa es 4,1 veces más lenta, cerca de la razón teórica de 3 de la tabla 1.

¿Dónde sí invierte algo EduFEM?

Solo en matrices pequeñas de tamaño fijo: el Jacobiano 2×2 de cada punto de Gauss, por la fórmula cerrada de cofactores (`fem/batch.py`, líneas 189–195), y la matriz de extrapolación de tensiones de los puntos de Gauss a los nodos, de 4×4 en Q4 y 9×9 en Q9, que es una constante del tipo de elemento (`fem/stress.py`, línea 131). La matriz de rigidez no se invierte nunca.

3. Factorización LU

3.1. La eliminación de Gauss, escrita como un producto

Resolver $\mathbf{Ax} = \mathbf{b}$ por eliminación de Gauss [1, 2] es anular, columna por columna, los coeficientes que quedan debajo de la diagonal. En el paso k el pivote es $a_{kk}^{(k)}$ y cada fila $i > k$ se modifica así:

$$\ell_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik} a_{kj}^{(k)} \quad (j = k, \dots, n), \quad (7)$$

lo que deja $a_{ik}^{(k+1)} = 0$. Tras $n - 1$ pasos queda una matriz triangular superior, $\mathbf{U} = \mathbf{A}^{(n)}$. Si los multiplicadores ℓ_{ik} se guardan en una matriz triangular inferior con unos en la diagonal, $\mathbf{L}$, se cumple

$$\boxed{\mathbf{A} = \mathbf{LU}} \quad (8)$$

La razón: el paso k equivale a multiplicar por la izquierda por $\mathbf{M}_k = \mathbf{I} - \ell_k \mathbf{e}_k^\top$, donde el vector ℓ_k contiene los multiplicadores de la columna k . Entonces $\mathbf{U} = \mathbf{M}_{n-1} \cdots \mathbf{M}_1 \mathbf{A}$, es decir, $\mathbf{A} = \mathbf{M}_1^{-1} \cdots \mathbf{M}_{n-1}^{-1} \mathbf{U}$. Cada inversa $\mathbf{M}_k^{-1} = \mathbf{I} + \ell_k \mathbf{e}_k^\top$ se obtiene cambiando un signo, y su producto es la matriz $\mathbf{L}$ con cada multiplicador en su lugar. *L sale gratis de la eliminación*: no hay que calcular nada más.

Algoritmo 1. LU sin pivoteo (Doolittle), sobre el propio arreglo $\mathbf{A}$.

```

para k = 1, ..., n-1
    para i = k+1, ..., n                (solo filas con  $a_{ik} \neq 0$ )
         $\ell[i,k] \leftarrow a[i,k] / a[k,k]$ 
        para j = k+1, ..., n            (solo columnas con  $a_{kj} \neq 0$ )
             $a[i,j] \leftarrow a[i,j] - \ell[i,k] \cdot a[k,j]$ 

```

Los dos comentarios entre paréntesis resumen toda la LU dispersa: si $a_{ik} = 0$ no hay multiplicador que calcular, y si $a_{kj} = 0$ no hay nada que actualizar.

3.2. Ejemplo 1, paso a paso: la cadena de resortes

Se factoriza $\mathbf{A} = \mathbf{K}/k$ de la ecuación (6) y al final se divide por k . El sistema es $\mathbf{K}\mathbf{u} = (0, 0, P)^\top$.

Paso 1: pivote $a_{11} = 2$

Fila 2: $\ell_{21} = \frac{-1}{2} = -\frac{1}{2}$; $a_{22} \leftarrow 2 - (-\frac{1}{2})(-1) = \frac{3}{2}$; $a_{23} \leftarrow -1 - (-\frac{1}{2})(0) = -1$.

Fila 3: $a_{31} = 0$, así que $\ell_{31} = 0$ y la fila no se toca. Aquí la dispersión ya ahorra trabajo.

$$\mathbf{A}^{(2)} = \begin{bmatrix} 2 & -1 & 0 \\ 0 & \frac{3}{2} & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

Paso 2: pivote $a_{22} = \frac{3}{2}$

Fila 3: $\ell_{32} = \frac{-1}{3/2} = -\frac{2}{3}$; $a_{33} \leftarrow 1 - (-\frac{2}{3})(-1) = \frac{1}{3}$.

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 \\ 0 & -\frac{2}{3} & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 2 & -1 & 0 \\ 0 & \frac{3}{2} & -1 \\ 0 & 0 & \frac{1}{3} \end{bmatrix}.$$

Comprobación de la fila 3 de $\mathbf{LU}$: $0 \cdot (2, -1, 0) - \frac{2}{3}(0, \frac{3}{2}, -1) + 1 \cdot (0, 0, \frac{1}{3}) = (0, -1, 1)$, que es la fila 3 de $\mathbf{A}$.

Paso 3: sustitución hacia adelante, $\mathbf{L}\mathbf{y} = \mathbf{b}$, con $\mathbf{b} = (0, 0, P/k)$

$$y_1 = 0; \quad y_2 = 0 - (-\frac{1}{2})y_1 = 0; \quad y_3 = \frac{P}{k} - (-\frac{2}{3})y_2 = \frac{P}{k}.$$

Paso 4: sustitución hacia atrás, $\mathbf{U}\mathbf{u} = \mathbf{y}$

$$u_3 = \frac{P/k}{1/3} = \frac{3P}{k}; \quad u_2 = \frac{0 - (-1)u_3}{3/2} = \frac{2P}{k}; \quad u_1 = \frac{0 - (-1)u_2}{2} = \frac{P}{k}.$$

Lo que muestra el ejemplo

- Comprobación física: cada resorte transmite P y se alarga P/k ; en efecto $u_1 - 0 = u_2 - u_1 = u_3 - u_2 = P/k$.
- Los ceros $a_{13} = a_{31} = 0$ de $\mathbf{K}$ siguen siendo ceros en $\mathbf{L}$ y $\mathbf{U}$: esta factorización no creó ningún coeficiente nuevo. Con n resortes en serie, $\mathbf{K}$ tiene $3n - 2$ no nulos, $\mathbf{L} + \mathbf{U}$ los mismos $3n - 2$ y $\mathbf{K}^{-1}$, n^2 .
- Los pivotes 2 , $\frac{3}{2}$ y $\frac{1}{3}$ son positivos y su producto es $\det \mathbf{A} = 1$.

3.3. Sustitución hacia adelante y hacia atrás

En general,

$$y_i = b_i - \sum_{j < i} \ell_{ij} y_j, \quad i = 1, 2, \dots, n, \quad (9)$$

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j > i} u_{ij} x_j \right), \quad i = n, n-1, \dots, 1. \quad (10)$$

Cada sustitución recorre una sola vez los coeficientes de su factor: $\approx n^2$ operaciones en denso y $\approx 2 \text{ nnz}$ en disperso. Es la parte barata.

3.4. ¿Por qué factorizar?

- **Separa lo caro de lo barato.** Factorizar se hace una vez; sustituir, una vez por cada vector de cargas. Con varios estados de carga, $\mathbf{L}$ y $\mathbf{U}$ se reutilizan (§7.8).
- **Diagnostica.** $\det \mathbf{K} = \prod_k u_{kk}$ (con pivoteo, salvo el signo de la permutación). Un pivote nulo delata una $\mathbf{K}_{ff}$ singular, es decir, un *mecanismo*: el modelo no está bien restringido. SuperLU lo detecta, SciPy devuelve NaN y EduFEM lo informa (`fem/solver.py`, líneas 134–139).
- **Conserva la dispersión** si las incógnitas se ordenan bien (§5), cosa que la inversa no puede hacer.

3.5. Pivoteo, y por qué una matriz SPD no lo necesita

La fórmula (7) divide por el pivote. Con $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ el primer pivote es cero y el algoritmo se detiene, aunque $\mathbf{A}$ es invertible; con $\begin{bmatrix} \epsilon & 1 \\ 1 & 1 \end{bmatrix}$ y ϵ diminuto, el multiplicador $1/\epsilon$ es enorme y amplifica el redondeo. La cura general es el **pivoteo parcial**: en el paso k se intercambia la fila k con la de mayor $|a_{ik}|$, y se obtiene $\mathbf{PA} = \mathbf{LU}$ con $|\ell_{ik}| \leq 1$.

Una matriz SPD no lo necesita, por dos razones:

1. **Los pivotes son positivos.** $u_{kk} = \det \mathbf{A}_k / \det \mathbf{A}_{k-1}$, donde $\mathbf{A}_k$ es la submatriz principal de orden k ; toda submatriz principal de una SPD es SPD y tiene determinante positivo. En el ejemplo 1, $\det \mathbf{A}_1 = 2$, $\det \mathbf{A}_2 = 3$ y $\det \mathbf{A}_3 = 1$, y los pivotes son $2/1$, $3/2$ y $1/3$.
2. **Los números no crecen.** Al eliminar, la diagonal de la parte que falta factorizar solo puede disminuir, $a_{ii}^{(k+1)} = a_{ii}^{(k)} - (a_{ik}^{(k)})^2 / a_{kk}^{(k)} \leq a_{ii}^{(k)}$, y en una SPD cada coeficiente está acotado por la diagonal, $|a_{ij}| \leq \sqrt{a_{ii} a_{jj}}$. Ningún coeficiente intermedio —ni de $\mathbf{U}$ — supera al mayor coeficiente diagonal de la matriz original, y la eliminación sin pivoteo es estable [3].

Esto permite elegir el orden de eliminación *solo* por la dispersión, sin preocuparse por la estabilidad (§5 y 6). SuperLU, que es un código general, pivotea igual, pero con un umbral que favorece la diagonal (§7.7).

3.6. Cholesky y $\mathbf{LDL}^\top$

Si $\mathbf{A}$ es simétrica, $\mathbf{U}$ no aporta información nueva: $\mathbf{U} = \mathbf{DL}^\top$ con $\mathbf{D} = \text{diag}(u_{11}, \dots, u_{nn})$, y entonces $\mathbf{A} = \mathbf{LDL}^\top$. En el ejemplo 1, $\mathbf{D} = \text{diag}(2, \frac{3}{2}, \frac{1}{3})$, y $\mathbf{DL}^\top$ reproduce $\mathbf{U}$ fila por fila; la fila 2, por ejemplo, es $\frac{3}{2} \cdot (0, 1, -\frac{2}{3}) = (0, \frac{3}{2}, -1)$. Si además $\mathbf{A}$ es definida positiva, $\mathbf{D}$ tiene raíz y $\mathbf{A} = \mathbf{GG}^\top$ con $\mathbf{G} = \mathbf{LD}^{1/2}$: es la **factorización de Cholesky**. Para el ejemplo 1,

$$\mathbf{G} = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ -1/\sqrt{2} & \sqrt{3/2} & 0 \\ 0 & -\sqrt{2/3} & 1/\sqrt{3} \end{bmatrix}.$$

Cholesky hace la mitad de las operaciones ($n^3/3$ en denso) y guarda un solo triángulo. EduFEM usa, sin embargo, la LU general de SuperLU por una razón práctica: SciPy no incluye una factorización de Cholesky dispersa —la de referencia, CHOLMOD, llega a Python con `scikit-sparse`, una dependencia binaria más que habría que empaquetar en el instalador—; la tesis la deja como recomendación. La diferencia es un factor constante, cercano a 2 en operaciones y en memoria, y no un cambio de orden de magnitud: con el ordenamiento simétrico de la sección 6 y pivotes diagonales, $\mathbf{U}$ tiene el mismo patrón que $\mathbf{L}^\top$. Medido con 33 024 GDL: $\text{nnz}(\mathbf{L}) = 2\,709\,732$ y $\text{nnz}(\mathbf{U}) = 2\,709\,732$.

4. Matrices dispersas: COO, CSR y CSC

4.1. Guardar solo lo que no es cero

Una matriz densa de orden n ocupa $8n^2$ bytes, ceros incluidos. Con 33 024 GDL libres, $\mathbf{K}_{ff}$ densa ocuparía 8,7 GB; en formato disperso ocupa 13 MB. Los formatos dispersos guardan solo los coeficientes no nulos, cada uno acompañado de la información necesaria para saber dónde va [4]. SciPy ofrece varios [21]; EduFEM usa tres.

4.2. COO: la lista de tripletes

El formato de coordenadas (*coordinate list*, COO) guarda tres arreglos de la misma longitud: `row`, `col` y `data`. El coeficiente número m es el triplete $(\text{row}[m], \text{col}[m], \text{data}[m])$. Dos propiedades lo hacen ideal para ensamblar:

- el orden de los tripletes no importa;
- puede haber tripletes repetidos en la misma posición (i, j) , y al convertir la matriz a otro formato (o al operar con ella) se suman.

La segunda propiedad es el ensamblaje (1): basta con listar todos los aportes elementales, cada uno en su posición global, y la conversión hace el «+=».

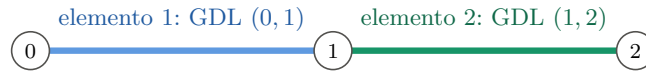


Figura 3: Ejemplo 4: barra de dos elementos con un GDL por nodo, numerados desde 0 como en el código. Cada elemento tiene $\mathbf{k}_e = k \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ con $k = EA/L$.

Ejemplo 4. Para la barra de la figura 3, cada elemento aporta sus cuatro coeficientes:

m	row	col	data	origen
0	0	0	k	elemento 1, $(\mathbf{k}_e)_{11}$
1	0	1	$-k$	elemento 1, $(\mathbf{k}_e)_{12}$
2	1	0	$-k$	elemento 1, $(\mathbf{k}_e)_{21}$
3	1	1	k	elemento 1, $(\mathbf{k}_e)_{22}$
4	1	1	k	elemento 2, $(\mathbf{k}_e)_{11}$
5	1	2	$-k$	elemento 2, $(\mathbf{k}_e)_{12}$
6	2	1	$-k$	elemento 2, $(\mathbf{k}_e)_{21}$
7	2	2	k	elemento 2, $(\mathbf{k}_e)_{22}$

Los tripletes 3 y 4 caen en la misma posición $(1, 1)$ y se suman: $k + k = 2k$. El nodo 1 pertenece a

los dos elementos y recibe la rigidez de ambos:

$$\mathbf{K} = k \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}.$$

4.3. CSR: filas comprimidas

El formato de filas comprimidas (*compressed sparse row*, CSR) guarda los valores fila por fila en `data`, la columna de cada valor en `indices`, y un arreglo `indptr` de longitud $n+1$ que marca dónde empieza cada fila: la fila i ocupa las posiciones `indptr[i], ..., indptr[i+1] - 1`. Para la barra:

```
indptr = [0, 2, 5, 7]
indices = [0, 1, 0, 1, 2, 1, 2]
data = [k, -k, -k, 2k, -k, -k, k]
```

Para leer la fila 1 se toman las posiciones 2 a 4: columnas 0, 1, 2 con valores $-k, 2k, -k$. Recorrer una fila cuesta tanto como sus no nulos, y la memoria es $\approx 12 \text{ nnz bytes}$ (8 del valor y 4 del índice) más $4(n+1)$ de `indptr`.

4.4. CSC: columnas comprimidas

El formato de columnas comprimidas (*compressed sparse column*, CSC) es lo mismo por columnas: `indptr` marca dónde empieza cada columna e `indices` guarda filas. Como $\mathbf{K}$ es simétrica, sus arreglos CSC son idénticos a los CSR —para la barra, el guion lo verifica—, pero el formato indica al programa cómo recorrerlos. SuperLU trabaja columna por columna (§7.5), así que su formato nativo es CSC.

4.5. Qué formato para qué

Formato	Bueno para	Dónde lo usa EduFEM
COO	construir: agregar tripletes en cualquier orden, con repetidos	<code>assemble_sparse</code> (<code>fem/batch.py</code>)
CSR	recorrer y extraer filas; producto $\mathbf{K}\mathbf{u}$	$\mathbf{K}$ global; submatriz $\mathbf{K}_{ff}$; reacciones $\mathbf{K}[r,:]\mathbf{u}$
CSC	recorrer y extraer columnas; algoritmos por columnas	$\mathbf{K}_{ff}$ justo antes de SuperLU (<code>K_red.tocsc()</code>)

4.6. El ensamblaje de EduFEM, línea por línea

Código 1: `fem/batch.py`: el ensamblaje completo de $\mathbf{K}$.

```
234 def assemble_sparse(dofs, ke, n_dof):
235     """Scatter local -> global como tripletes COO y conversion a CSR.
236
237     `coo_matrix` suma las entradas duplicadas en la misma (i, j): es
238     exactamente la suma de contribuciones elementales del MEF. K nunca se
239     materializa densa (memoria O(nnz)).
240     """
241     dofs = np.asarray(dofs, dtype=np.intp)
242     n_dof_e = dofs.shape[1] if dofs.ndim == 2 else 0
243     rows = np.repeat(dofs, n_dof_e, axis=1).ravel() # [i0, i0, ..., i1, i1, ...]
244     cols = np.tile(dofs, (1, n_dof_e)).ravel()      # [j0, j1, ..., j0, j1, ...]
245     data = np.asarray(ke, dtype=float).ravel()
246     return coo_matrix((data, (rows, cols)), shape=(n_dof, n_dof)).tocsr()
```

`dofs` es un arreglo de $e \times 2n_e$ con los GDL globales de cada elemento ($2n_e = 8$ en Q4 y 18 en Q9) y `ke` es el lote de rigideces, de $e \times 2n_e \times 2n_e$. Con la barra, `dofs = [[0, 1], [1, 2]]` y:

- `np.repeat(dofs, 2, axis=1).ravel()` repite cada GDL tantas veces como columnas tiene k_e : `[0,0,1,1, 1,1,2,2]`, la columna `row` de la tabla anterior;
- `np.tile(dofs, (1, 2)).ravel()` repite la lista entera de GDL del elemento una vez por fila de k_e : `[0,1,0,1, 1,2,1,2]`, la columna `col`;
- `ke.ravel()` aplanar las matrices elementales fila por fila: `[k,-k,-k,k, k,-k,-k,k]`, la columna `data`.

El triplete $m = e(2n_e)^2 + a(2n_e) + b$ lleva $(k_e)_{ab}$ a la posición global $(dofs[e, a], dofs[e, b])$: es el operador $A_e^T k_e A_e$ de la ecuación (1) sin construir ninguna A_e . Finalmente, `coo_matrix((data, (rows, cols)), ...)` arma la matriz COO y `.tocsr()` la convierte a CSR sumando los repetidos. El guion verifica que esta misma función, aplicada a la barra, devuelve exactamente los arreglos `indptr`, `indices` y `data` de arriba.

Con la membrana de Cook de 8×8 elementos Q9 hay 64 elementos $\times 18^2 = 20\,736$ tripletes; al sumar los 3836 repetidos quedan 16\,900 coeficientes en K . Con 64×64 elementos, 1\,327\,104 tripletes se reducen a 1\,052\,676 coeficientes.

Quién hace qué en el ensamblaje

NumPy calcula el lote de k_e (`einsum`, `matmul` en `stiffness_batch`) y arma `rows`, `cols` y `data` con `repeat`, `tile` y `ravel`: bucles en C, sin un bucle de Python por elemento. **SciPy** (`scipy.sparse`) construye la matriz COO y la convierte a CSR, también en código compilado; la suma de repetidos ocurre ahí. La matriz nunca existe en forma densa.

5. El llenado: por qué importa el orden de las incógnitas

5.1. Qué es el llenado

Se llama **llenado** (*fill-in*) a los coeficientes que son cero en la matriz y dejan de serlo en L o en U . Aparecen por la actualización (7): si $a_{ij} = 0$ pero $\ell_{ik} \neq 0$ y $a_{kj} \neq 0$, entonces $a_{ij} - \ell_{ik}a_{kj} \neq 0$. Cada coeficiente de llenado ocupa memoria y, peor, participa en todas las actualizaciones posteriores. En la cadena de resortes no hubo llenado; el ejemplo siguiente muestra que la misma física puede generar mucho o nada, según el orden.

5.2. Ejemplo 2: una estrella

Un nodo central (1) está unido por resortes de rigidez 1 a tres nodos periféricos (2, 3 y 4), y cada nodo está además anclado a tierra: el central con un resorte de rigidez 1 y los periféricos con resortes de rigidez 2. Con fuerzas $F = (1, 2, 2, 2)$:

$$K = \begin{bmatrix} 4 & -1 & -1 & -1 \\ -1 & 3 & 0 & 0 \\ -1 & 0 & 3 & 0 \\ -1 & 0 & 0 & 3 \end{bmatrix}, \quad F = \begin{bmatrix} 1 \\ 2 \\ 2 \\ 2 \end{bmatrix}, \quad u = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

(Todos los nodos se desplazan 1: los resortes entre nodos no se estiran y cada anclaje toma su propia fuerza.) K tiene 10 coeficientes no nulos de 16.

Orden A: el nodo central primero

Paso 1, pivote 4: $\ell_{21} = \ell_{31} = \ell_{41} = -\frac{1}{4}$. Para $i, j \in \{2, 3, 4\}$, $a_{ij} \leftarrow a_{ij} - (-\frac{1}{4})(-1) = a_{ij} - \frac{1}{4}$: la diagonal pasa de 3 a $\frac{11}{4}$ y **los seis ceros se vuelven $-\frac{1}{4}$** :

$$\mathbf{A}^{(2)} = \begin{bmatrix} 4 & -1 & -1 & -1 \\ 0 & \frac{11}{4} & -\frac{1}{4} & -\frac{1}{4} \\ 0 & -\frac{1}{4} & \frac{11}{4} & -\frac{1}{4} \\ 0 & -\frac{1}{4} & -\frac{1}{4} & \frac{11}{4} \end{bmatrix}.$$

Paso 2, pivote $\frac{11}{4}$: $\ell_{32} = \ell_{42} = \frac{-1/4}{11/4} = -\frac{1}{11}$; $a_{33} \leftarrow \frac{11}{4} - \frac{1}{44} = \frac{30}{11}$; $a_{34} \leftarrow -\frac{1}{4} - \frac{1}{44} = -\frac{3}{11}$ (y lo mismo en la fila 4).

Paso 3, pivote $\frac{30}{11}$: $\ell_{43} = \frac{-3/11}{30/11} = -\frac{1}{10}$; $a_{44} \leftarrow \frac{30}{11} - \frac{3}{110} = \frac{27}{10}$.

$$\mathbf{L} = \begin{bmatrix} 1 & & & \\ -\frac{1}{4} & 1 & & \\ -\frac{1}{4} & -\frac{1}{11} & 1 & \\ -\frac{1}{4} & -\frac{1}{11} & -\frac{1}{10} & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 4 & -1 & -1 & -1 \\ & \frac{11}{4} & -\frac{1}{4} & -\frac{1}{4} \\ & & \frac{30}{11} & -\frac{3}{11} \\ & & & \frac{27}{10} \end{bmatrix}.$$

$\mathbf{L} + \mathbf{U}$ tiene los 16 coeficientes llenos: **6 de llenado** (en naranja).

Orden B: el nodo central al final

Se renumeran los nodos como $(2, 3, 4, 1)$, es decir, se factoriza $\mathbf{PKP}^\top$:

$$\mathbf{PKP}^\top = \begin{bmatrix} 3 & 0 & 0 & -1 \\ 0 & 3 & 0 & -1 \\ 0 & 0 & 3 & -1 \\ -1 & -1 & -1 & 4 \end{bmatrix}.$$

Paso 1, pivote 3: solo la fila 4 tiene un no nulo en la columna 1, $\ell_{41} = -\frac{1}{3}$, y $a_{44} \leftarrow 4 - (-\frac{1}{3})(-1) = \frac{11}{3}$. La fila 1 tiene ceros en las columnas 2 y 3, así que no se crea nada.

Pasos 2 y 3, iguales: $\ell_{42} = \ell_{43} = -\frac{1}{3}$, y a_{44} baja a $\frac{10}{3}$ y después a 3.

$$\mathbf{L} = \begin{bmatrix} 1 & & & \\ 0 & 1 & & \\ 0 & 0 & 1 & \\ -\frac{1}{3} & -\frac{1}{3} & -\frac{1}{3} & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 3 & 0 & 0 & -1 \\ & 3 & 0 & -1 \\ & & 3 & -1 \\ & & & 3 \end{bmatrix}.$$

$\mathbf{L} + \mathbf{U}$ tiene 10 coeficientes, los mismos de $\mathbf{K}$: **ningún llenado**.

Los dos órdenes resuelven el mismo problema. En el orden B, la sustitución hacia adelante con $\mathbf{PF} = (2, 2, 2, 1)$ da $\mathbf{y} = (2, 2, 2, 3)$, y hacia atrás, $u = 1$ en los cuatro nodos. Los determinantes también coinciden: $4 \cdot \frac{11}{4} \cdot \frac{30}{11} \cdot \frac{27}{10} = 81 = 3^4$.

5.3. Renumerar es permutar

Cambiar el orden de las incógnitas equivale a multiplicar por una matriz de permutación $\mathbf{P}$ (la identidad con las filas reordenadas), que cumple $\mathbf{P}^\top \mathbf{P} = \mathbf{I}$:

$$\mathbf{K}\mathbf{u} = \mathbf{F} \iff (\mathbf{PKP}^\top)(\mathbf{P}\mathbf{u}) = \mathbf{P}\mathbf{F}. \quad (11)$$

La física no cambia: solo cambia en qué orden se eliminan las ecuaciones. En EduFEM la numeración de los nodos es la que haya dejado el usuario —el orden de los clics en el lienzo, el del archivo DXF importado, y en Q9 los nodos intermedios que `expand_q4_to_q9` agrega al final—, así que el solucionador no puede confiar en ella. Reordena por su cuenta.

5.4. El grafo de la matriz

La herramienta para razonar sobre el llenado es el grafo de la matriz [6, 4]: un vértice por incógnita y una arista $\{i, j\}$ por cada coeficiente $K_{ij} \neq 0$ con $i \neq j$. En el MEF ese grafo es, esencialmente, la *conectividad de la malla*: dos nodos son vecinos si comparten un elemento.

Regla de eliminación en el grafo [7, 8]

Eliminar la incógnita v equivale, en el grafo, a **unir entre sí a todos los vecinos de v** (forman un bloque totalmente conectado) y quitar v . Las aristas nuevas son exactamente el llenado. Si v tiene d vecinos, se crean como mucho $d(d-1)/2$ aristas, y la eliminación cuesta del orden de d^2 operaciones.

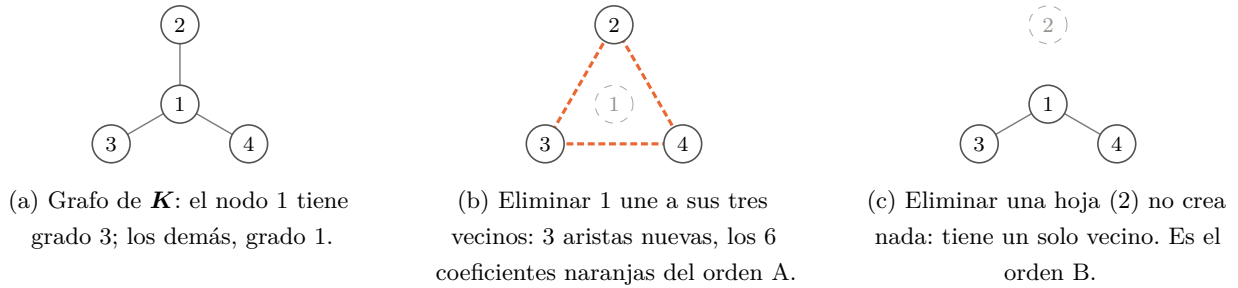


Figura 4: El ejemplo 2 en el lenguaje de grafos. Líneas grises: coeficientes de K ; naranja discontinuo: llenado; círculo punteado: incógnita ya eliminada.

5.5. ¿Cuál es el mejor orden?

Encontrar el orden de menor llenado es un problema NP-completo [12]: con miles de incógnitas no se puede buscar el óptimo, así que se usan heurísticas. Hay tres familias clásicas [5]:

- **Reducción del ancho de banda** (Cuthill–McKee y su versión inversa, RCM): numera de modo que los no nulos queden cerca de la diagonal; el llenado queda confinado dentro de la banda. EduFEM lo probó y lo retiró el 2026-09-06, porque el mínimo grado ya daba la ganancia grande (`docs/convenciones/roadmap-fem.md`, ítem 3).
- **Mínimo grado**: una regla local y voraz que ataca el llenado directamente. Es la que usa EduFEM (§6).
- **Dissección anidada**: parte la malla recursivamente con separadores y los numera al final; en mallas planas alcanza el orden de magnitud óptimo, $O(n \log n)$ de llenado y $O(n^{3/2})$ operaciones [11].

6. Ordenamiento de mínimo grado sobre $K^\top + K$

6.1. La regla

El **grado** de un vértice es su número de vecinos en el grafo *actual* de eliminación; en la matriz, es el número de no nulos fuera de la diagonal en su fila de la parte que falta factorizar. Como eliminar un vértice de grado d crea hasta $d(d-1)/2$ aristas y cuesta $\approx d^2$ operaciones, la regla del mínimo grado [9, 10] es simple: *eliminar siempre un vértice de grado mínimo*, actualizar el grafo y repetir.

Algoritmo 2. Mínimo grado.

```

G ← grafo de K
para k = 1, ..., n
    v ← un vértice de G con grado mínimo    (empates: el de menor número)
    π(k) ← v                                (v será la incógnita k-ésima)
    unir entre sí a todos los vecinos de v    (aristas de llenado)
    quitar v de G

```

El resultado es una permutación π , es decir, una matriz $\mathbf{P}$ para la ecuación (11). El algoritmo trabaja *solo con el patrón* de la matriz, nunca con sus valores, y termina antes de que empiece la factorización numérica.

6.2. Ejemplo 3, paso a paso: una malla de 2×2 elementos Q4

La malla tiene 9 nodos numerados por filas (1–2–3 abajo, 7–8–9 arriba). Cada elemento une a sus cuatro nodos entre sí, así que el grafo tiene 20 aristas: cada nodo de esquina tiene grado 3, cada nodo de lado, grado 5, y el central, grado 8. Para que el ejemplo se pueda seguir a mano se trabaja con los nodos; la sección 6.3 explica por qué con dos GDL por nodo el resultado es el mismo. La figura 5 y la tabla 3 siguen el algoritmo 2 paso a paso.

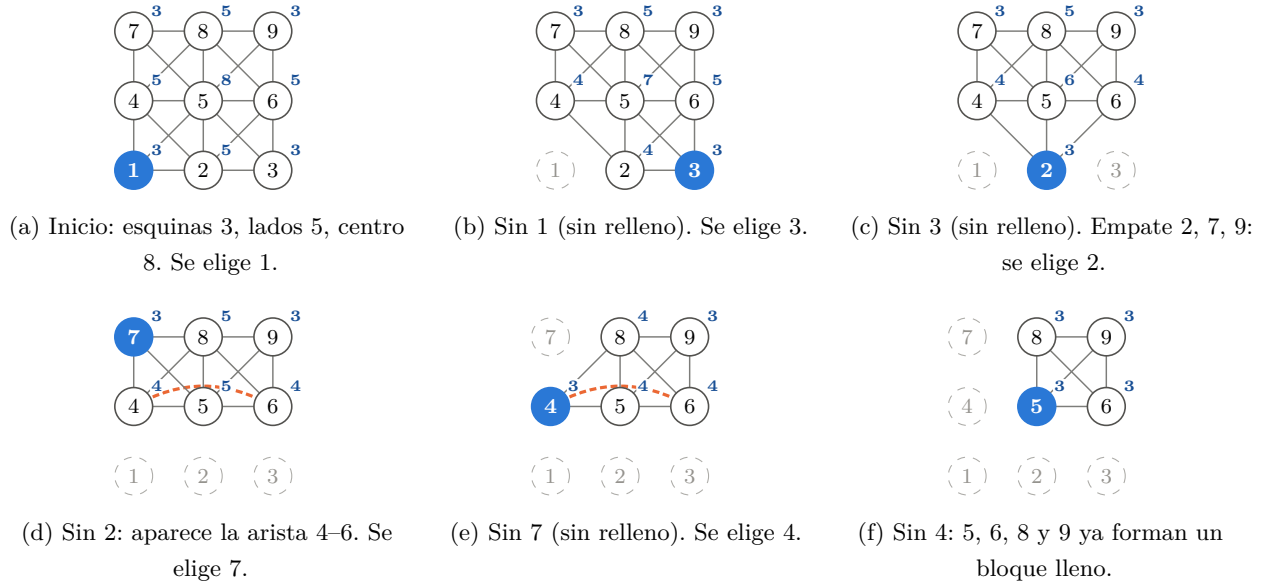


Figura 5: Mínimo grado sobre el grafo de la malla 2×2 . Número azul junto a cada nodo: su grado actual. Nodo azul: el que se elimina en ese paso. Círculo punteado: nodo ya eliminado. Naranja discontinuo: arista de llenado.

Tabla 3: Los pasos de la figura 5. «Grados» lista *nodo:grado* antes de elegir.

Paso	Grados	Elige	Vecinos	Llenado
1	1:3, 2:5, 3:3, 4:5, 5:8, 6:5, 7:3, 8:5, 9:3	1	2, 4, 5	—
2	2:4, 3:3, 4:4, 5:7, 6:5, 7:3, 8:5, 9:3	3	2, 5, 6	—
3	2:3, 4:4, 5:6, 6:4, 7:3, 8:5, 9:3	2	4, 5, 6	4–6
4	4:4, 5:5, 6:4, 7:3, 8:5, 9:3	7	4, 5, 8	—
5	4:3, 5:4, 6:4, 8:4, 9:3	4	5, 6, 8	—
6	5:3, 6:3, 8:3, 9:3	5	6, 8, 9	—
7–9	6:2, 8:2, 9:2 → ...	6, 8, 9	—	—

El orden resultante, 1, 3, 2, 7, 4, 5, 6, 8, 9, produce **una sola arista de llenado**, 4–6. El guion comprueba por fuerza bruta, sobre las $9! = 362\,880$ numeraciones posibles, que 1 es el mínimo

alcanzable: en este caso la heurística encontró el óptimo, algo que en general no está garantizado. La figura 6 compara el resultado con otros dos órdenes.

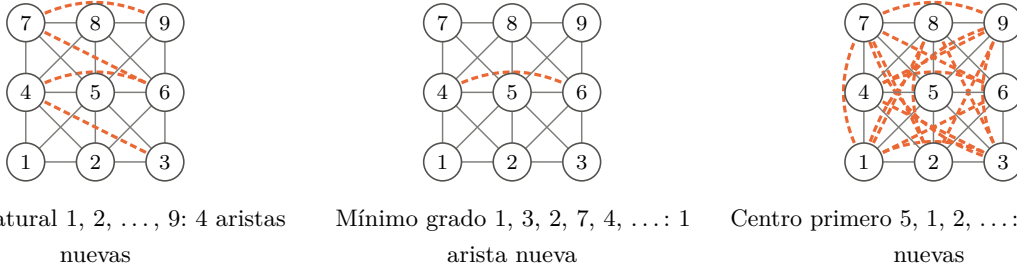


Figura 6: Grafo final —aristas originales más llenado— para tres órdenes de eliminación de la misma malla.

La lógica del mínimo grado se ve en la figura: los nodos de esquina salen primero y gratis, porque todos sus vecinos pertenecen al mismo elemento y ya están conectados entre sí; los nodos más conectados —el centro, que actúa como separador de la malla— quedan para el final, cuando ya no tienen a quién conectar. En cambio, eliminar primero el centro une a sus ocho vecinos entre sí y produce 16 aristas nuevas.

6.3. De nodos a GDL

Cada nodo tiene dos GDL, u_x y u_y , y los dos tienen exactamente los mismos vecinos: todos los GDL de los nodos con que comparten algún elemento, más el otro GDL del propio nodo. Dos vértices con la misma vecindad se llaman **indistinguibles**, y un buen algoritmo de mínimo grado los detecta y los elimina juntos. Por eso el grafo de 18 GDL se comporta como el de 9 nodos con bloques de 2×2 : cada arista de llenado entre nodos cuesta un bloque de 2×2 en $\mathbf{L}$ y otro en $\mathbf{U}$. Esa misma estructura por pares es la que SuperLU aprovecha como supernodos (§7.6).

6.4. Mínimo grado múltiple (MMD), el que usa SuperLU

SuperLU calcula el ordenamiento con **GENMMD**, la implementación de Liu del *mínimo grado múltiple* [13, 14]. Mejora la regla básica en tres puntos:

1. **Eliminación múltiple:** en cada ronda elimina un conjunto de vértices de grado mínimo que no son vecinos entre sí, y recién después actualiza los grados, que es lo más caro del algoritmo.
2. **Eliminación en masa:** los vértices indistinguibles se tratan como uno solo (los u_x , u_y de un nodo).
3. **Grado externo:** cuenta solo los vecinos que no pertenecen al propio grupo de indistinguibles, lo que mide mejor el llenado real.

6.5. ¿Por qué sobre $\mathbf{K}^\top + \mathbf{K}$?

El mínimo grado necesita un grafo *no dirigido*: la arista $\{i, j\}$ tiene que existir tanto si $a_{ij} \neq 0$ como si $a_{ji} \neq 0$. SuperLU es una biblioteca para matrices generales, que pueden no ser simétricas, y por eso aplica el mínimo grado al patrón de $\mathbf{A}^\top + \mathbf{A}$: el menor patrón simétrico que contiene al de $\mathbf{A}$. Para la rigidez,

$$(\mathbf{K}^\top + \mathbf{K})_{ij} = K_{ji} + K_{ij} = 2K_{ij}, \quad (12)$$

que tiene exactamente el patrón de $\mathbf{K}$. En EduFEM, entonces, **MMD_AT_PLUS_A** es el **mínimo grado sobre el grafo de la propia malla**. Solo se usa la estructura (el código de SuperLU arma el patrón de $\mathbf{A}^\top + \mathbf{A}$ sin sumar valores), de modo que no hay cancelaciones que puedan borrar una arista.

SciPy expone cuatro ordenamientos de SuperLU con el parámetro `permc_spec`:

permc_spec	Qué hace
NATURAL	No reordena: usa la numeración que traiga la matriz.
MMD_ATA	Mínimo grado múltiple sobre el patrón de $\mathbf{A}^\top \mathbf{A}$.
COLAMD	Mínimo grado <i>aproximado</i> por columnas [16]: el mismo objetivo que MMD_ATA, pero sin formar $\mathbf{A}^\top \mathbf{A}$. Es el valor por defecto de SciPy.
MMD_AT_PLUS_A	Mínimo grado múltiple sobre el patrón de $\mathbf{A}^\top + \mathbf{A}$. Es el que usa EduFEM (SOLVER_PERMC_SPEC).

¿Por qué existen las variantes sobre $\mathbf{A}^\top \mathbf{A}$? Para matrices no simétricas, el pivoteo parcial decide los intercambios de filas *durante* la factorización, según los valores, y no se puede predecir. El patrón del factor de Cholesky de $\mathbf{A}^\top \mathbf{A}$ acota la estructura de $\mathbf{U}$ sea cual sea la secuencia de intercambios [15], así que ordenar las columnas para $\mathbf{A}^\top \mathbf{A}$ da un orden seguro frente a cualquier pivoteo. El precio es un modelo más grueso. Para la rigidez, $\mathbf{K}^\top \mathbf{K} = \mathbf{K}^2$, cuyo grafo une GDL que están a *dos* elementos de distancia: en la malla Q4, las filas pasan de un máximo de 18 no nulos en $\mathbf{K}_{ff}$ a 50 en $\mathbf{K}_{ff}^\top \mathbf{K}_{ff}$ (2,7 veces más no nulos en total; en Q9, 3,9 veces). Como en $\mathbf{K}$ SPD el pivoteo casi nunca se aparta de la diagonal (§7.7), ese seguro no hace falta, y el modelo simétrico $\mathbf{K}^\top + \mathbf{K}$ ordena mejor.

6.6. Lo que se mide en EduFEM

Las tablas 4 y 5 comparan los cuatro ordenamientos sobre la membrana de Cook, y la figura 7 muestra el llenado en la malla de 8×8 .

Tabla 4: Llenado según el ordenamiento (membrana de Cook, Q9): $\text{nnz}(\mathbf{L} + \mathbf{U}) / \text{nnz}(\mathbf{K}_{ff})$, donde $\text{nnz}(\mathbf{L} + \mathbf{U}) = \text{nnz}(\mathbf{L}) + \text{nnz}(\mathbf{U}) - n$ (la diagonal unitaria de $\mathbf{L}$ no cuenta). Las dos últimas columnas son de MMD_AT_PLUS_A: coeficientes de $\mathbf{L} + \mathbf{U}$ y operaciones de la factorización, contadas sobre la estructura de los factores (apéndice A). «—»: no medido (tardaría varios minutos).

n	$\text{nnz}(\mathbf{K}_{ff})$	$\text{nnz}(\mathbf{L} + \mathbf{U}) / \text{nnz}(\mathbf{K}_{ff})$				Con MMD_AT_PLUS_A	
		NATURAL	COLAMD	MMD_ATA	MMD_AT_PLUS_A	$\text{nnz}(\mathbf{L} + \mathbf{U})$	Operaciones
544	15 600	14,9	2,6	2,6	1,7	27 104	$7,9 \times 10^5$
2112	63 984	53,5	4,5	4,2	2,4	156 040	$8,0 \times 10^6$
4704	145 136	116,0	6,4	5,5	3,1	444 760	$3,6 \times 10^7$
8320	259 056	202,6	6,8	6,5	3,6	944 056	$1,0 \times 10^8$
33 024	1 042 416	—	11,0	10,5	5,2	5 386 440	$1,1 \times 10^9$

Tabla 5: Tiempo de `spsolve` (factorización y sustitución, en segundos) según el ordenamiento, sobre el mismo $\mathbf{K}_{ff}$ en CSC. Mejor de 15 corridas (7 con 33 024 GDL; 1 en los casos de más de 10 s), en un equipo en uso normal. Última columna: cuántas veces más tarda COLAMD que MMD_AT_PLUS_A.

n	NATURAL	COLAMD	MMD_ATA	MMD_AT_PLUS_A	COLAMD/MMD_AT_PLUS_A
544	0,019	0,0025	0,0041	0,0020	1,3
2112	1,7	0,018	0,029	0,013	1,4
4704	13	0,075	0,078	0,027	2,8
8320	150	0,22	0,31	0,13	1,6
33 024	—	2,9	3,3	1,1	2,6

Qué dicen las mediciones

- MMD_AT_PLUS_A produce el menor llenado en todas las mallas: 5,2 veces $\text{nnz}(\mathbf{K}_{ff})$ con 33 024 GDL, contra 11,0 de COLAMD y 10,5 de MMD_ATA. Con ese tamaño, COLAMD hace 2,6 veces más operaciones y tarda 2,9 s frente a 1,1 s: 2,6 veces más.

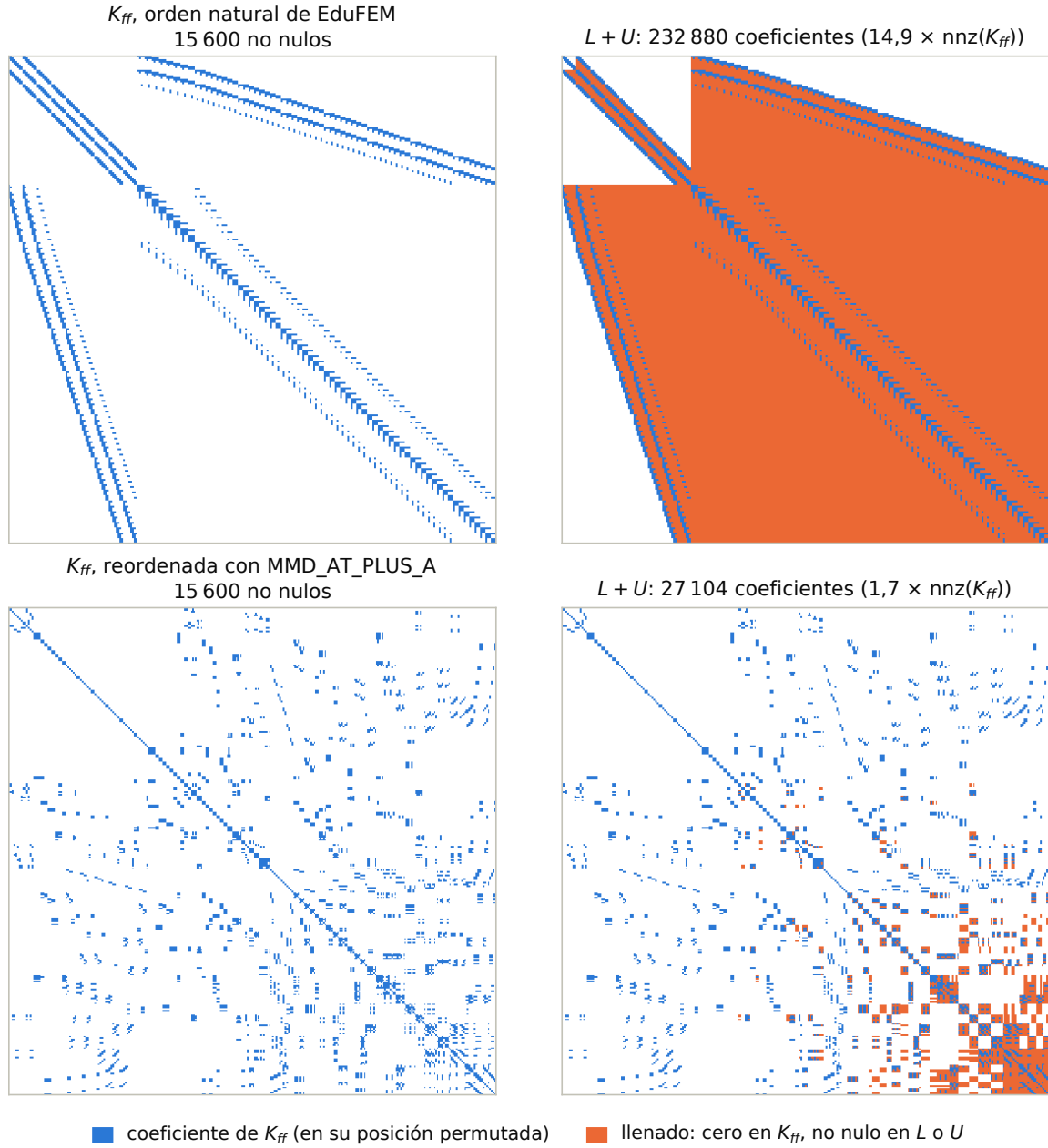


Figura 7: Patrón de la matriz que SuperLU factoriza y de sus factores, membrana de Cook 8×8 Q9 (544 GDL libres). Arriba: numeración de EduFEM, sin reordenar; los bloques alejados de la diagonal son los nodos intermedios y centrales de los Q9, que la expansión de Q4 a Q9 numera al final. Abajo: la misma matriz reordenada con MMD_AT_PLUS_A; la parte densa de $L + U$ se concentra en el ángulo inferior derecho, donde quedaron los separadores.

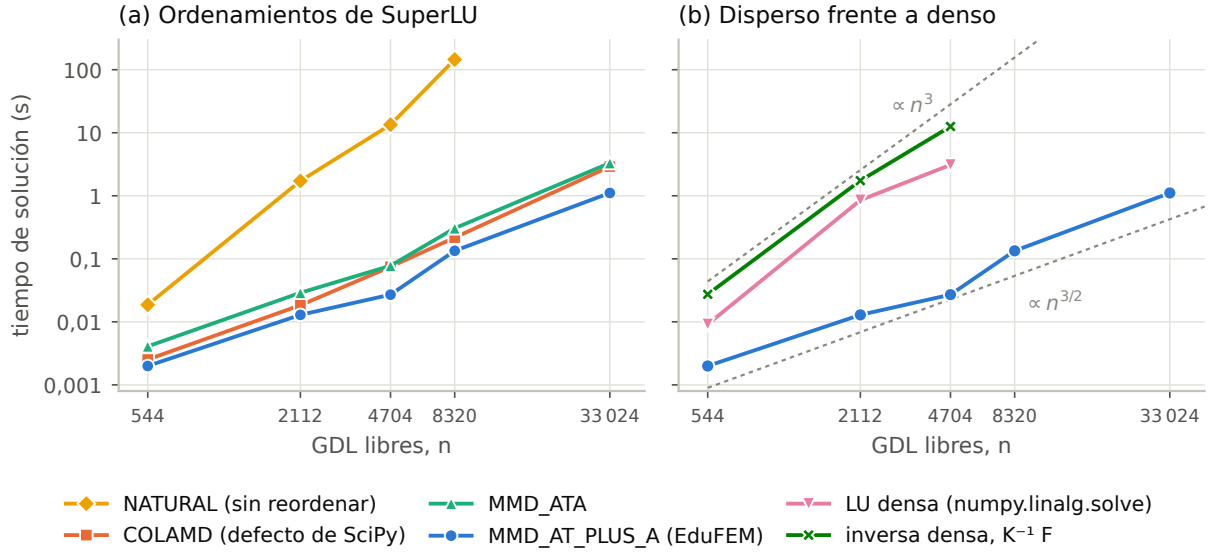


Figura 8: Tiempo de solución frente al número de GDL libres (ejes logarítmicos; los valores exactos están en las tablas 2 y 5). (a) Los cuatro ordenamientos de SuperLU. (b) SuperLU con MMD_AT_PLUS_A frente a los dos caminos densos; las líneas punteadas marcan pendientes de referencia.

- Sin reordenar, el llenado se dispara en Q9: con 8320 GDL, $\mathbf{L} + \mathbf{U}$ tiene 52 485 872 coeficientes (202,6 veces $\mathbf{K}_{ff}$), la factorización hace 2110 veces más operaciones que con MMD_AT_PLUS_A y la solución tarda 150 s, 1084 veces más. La causa es la numeración: la expansión a Q9 agrega los nodos nuevos al final, lejos de sus vecinos.
- El llenado con MMD_AT_PLUS_A crece despacio: al multiplicar n por 61, la razón $\text{nnz}(\mathbf{L} + \mathbf{U}) / \text{nnz}(\mathbf{K}_{ff})$ pasa de 1,7 a 5,2, coherente con el $O(n \log n)$ de las mallas planas. Las operaciones crecieron 1450 veces, como $n^{1,77}$: algo más que el $n^{3/2}$ de la disección anidada, que el mínimo grado no garantiza, y muchísimo menos que el n^3 denso. El tiempo medido creció menos, 556 veces, porque la velocidad de cálculo sube con el tamaño: de 0,39 a 1,0 GFLOP/s, a medida que los supernodos crecen y el costo fijo de cada llamada pierde peso.

Con elementos Q4 y numeración por filas, la numeración natural ya es razonable —es una numeración de banda—, y la ventaja de MMD_AT_PLUS_A es menor pero se mantiene (tabla 6). La conclusión práctica es que el ordenamiento independiza al solucionador de la numeración que haya elegido el usuario.

Tabla 6: Llenado $\text{nnz}(\mathbf{L} + \mathbf{U}) / \text{nnz}(\mathbf{K}_{ff})$ con elementos Q4 (membrana de Cook), en mallas con los mismos GDL que las Q9 de la tabla 4. Última columna: la numeración natural de Q9 con igual n , para comparar.

n	$\text{nnz}(\mathbf{K}_{ff})$	NATURAL	COLAMD	MMD_ATA	MMD_AT_PLUS_A	NATURAL en Q9
544	9016	4,0	3,5	3,6	3,0	14,9
2112	36 472	7,5	5,6	6,2	4,5	53,5
8320	146 680	14,6	11,1	8,7	6,5	202,6

7. SuperLU por dentro

7.1. Qué es

SuperLU es una biblioteca en C para la factorización LU de matrices dispersas generales con pivoteo parcial, desarrollada por Demmel, Eisenstat, Gilbert, Li y Liu [19, 20]. SciPy [21] incluye su versión secuencial compilada dentro de `scipy.sparse.linalg` y la expone con dos funciones, `spsolve` y `splu`. El «*Super*» del nombre viene de los *supernodos* (§7.6).

7.2. La factorización que calcula

SuperLU calcula

$$\boxed{P_r A P_c = L U} \quad (13)$$

donde P_c permuta las columnas según el ordenamiento (§6) y P_r permuta las filas según el pivoteo. Así es como lo documenta SciPy (`Pr @ A @ Pc = L @ U`). Para resolver $Ax = b$:

$$Ax = b \iff \underbrace{P_r A P_c}_{LU} \underbrace{P_c^\top x}_z = P_r b, \quad (14)$$

es decir, cuatro pasos: reordenar $\hat{b} = P_r b$; sustituir hacia adelante $Ly = \hat{b}$; sustituir hacia atrás $Uz = y$; y deshacer el orden, $x = P_c z$. Si el pivoteo elige siempre el coeficiente diagonal, $P_r = P_c^\top$ y la ecuación (13) es la renumeración simétrica $PKP^\top$ de la sección 5.

7.3. Las cuatro fases

`spsolve` llama al controlador simple de SuperLU, `dgssv`, que encadena cuatro fases:

1. **Ordenamiento** (`get_perm_c`): calcula P_c según `permc_spec`. Con `MMD_AT_PLUS_A`, arma el patrón de $A^\top + A$ y llama a `GENMMD`.
2. **Preordenamiento** (`sp_preorder`): aplica P_c , calcula el *árbol de eliminación* de las columnas y lo recorre en postorden, que reordena levemente P_c sin cambiar el llenado, de modo que las columnas de cada rama y de cada supernodo queden contiguas.
3. **Factorización numérica** (`dgstrf`): columna por columna, de izquierda a derecha, agrupando columnas en paneles y supernodos, con pivoteo parcial por umbral. Como el pivoteo puede cambiar la estructura, el cálculo simbólico (qué coeficientes serán no nulos) y el numérico se hacen juntos.
4. **Sustituciones** (`dgstrs`): las dos resoluciones triangulares, con las permutaciones.

7.4. El árbol de eliminación

El padre de la columna j en el árbol de eliminación es la fila del primer no nulo de L debajo de la diagonal:

$$\text{padre}(j) = \min\{i > j : \ell_{ij} \neq 0\}. \quad (15)$$

La columna j solo puede modificar a sus ancestros, así que dos ramas distintas del árbol no dependen una de otra [17]. En la estrella (§5.2), con el orden B, $\ell_{41}, \ell_{42}, \ell_{43} \neq 0$: las columnas 1, 2 y 3 tienen por padre a la 4, y el árbol es la propia estrella, con tres hojas independientes. Con el orden A, el padre de 1 es 2, el de 2 es 3 y el de 3 es 4: una cadena en la que cada columna espera a la anterior. Los buenos órdenes producen árboles bajos y frondosos; los malos, cadenas.

7.5. Factorización por columnas, «mirando a la izquierda»

SuperLU calcula la columna j de $\mathbf{L}$ y $\mathbf{U}$ usando las columnas ya terminadas, que están a su izquierda. Con $\mathbf{a}_j$ la columna j de la matriz ya permutada:

$$\mathbf{u}_{1:j-1,j} = \mathbf{L}_{1:j-1,1:j-1}^{-1} \mathbf{a}_{1:j-1,j} \quad (\text{resolución triangular dispersa}) \quad (16)$$

$$\mathbf{w} = \mathbf{a}_{j:n,j} - \mathbf{L}_{j:n,1:j-1} \mathbf{u}_{1:j-1,j} \quad (\text{actualización con las columnas previas}) \quad (17)$$

$$u_{jj} = w_p, \quad \ell_{j+1:n,j} = \mathbf{w}_{\text{resto}}/u_{jj} \quad (\text{pivote } w_p \text{ elegido por umbral, §7.7}) \quad (18)$$

(La «inversa» de $\mathbf{L}$ en la primera línea es notación: se resuelve por sustitución, ecuación (9).) La clave dispersa está en la primera línea: $\mathbf{u}_{1:j-1,j}$ solo tiene no nulos en las posiciones alcanzables, en el grafo de $\mathbf{L}$, desde los no nulos de $\mathbf{a}_j$, y una búsqueda en profundidad las encuentra en un tiempo proporcional a las operaciones útiles [18]. Nunca se recorre un cero.

7.6. Supernodos

Un **supernodo** es un grupo de columnas contiguas de $\mathbf{L}$ con la misma estructura debajo de su bloque diagonal. SuperLU guarda cada supernodo como un bloque denso rectangular, y entonces la actualización con todo el supernodo se convierte en operaciones densas —una resolución triangular y un producto matriz–vector—, y al procesar varias columnas j a la vez (un panel), en productos matriz–matriz, que aprovechan la memoria caché del procesador mucho mejor que operar coeficiente por coeficiente [19].

En el MEF los supernodos aparecen solos: los dos GDL de un nodo son indistinguibles (§6.3) y forman al menos un supernodo de dos columnas. Medido con MMD_AT_PLUS_A en la malla de 33 024 GDL: el tamaño medio de los supernodos fundamentales de $\mathbf{L}$ es de 4,0 columnas (unos dos nodos), el 100 % tiene al menos dos columnas y el mayor tiene 566: es el bloque denso del último separador, el ángulo inferior derecho de la figura 7.

7.7. Pivoteo por umbral

En la columna j , con $\mathbf{w}$ el vector de candidatos a pivote, SuperLU acepta el coeficiente diagonal si

$$|w_{\text{diag}}| \geq u \cdot \max_i |w_i|, \quad u \in [0, 1], \quad (19)$$

y si no, toma el de mayor módulo. $u = 1$ es el pivoteo parcial clásico (la diagonal se acepta solo si es la mayor) y $u = 0$ acepta siempre la diagonal si no es nula. `spsolve` no expone este parámetro y usa el valor por defecto de SuperLU, $u = 1$; `splu` lo expone como `diag_pivot_thresh`.

Para una $\mathbf{K}_{ff}$ SPD el pivote diagonal es estable (§3.5), pero con $u = 1$ SuperLU puede igualmente intercambiar filas cuando en alguna columna un coeficiente fuera de la diagonal supera al diagonal, algo posible en una SPD: en $\begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}$ la primera columna tiene $2 > 1$. Medido con 33 024 GDL: $\mathbf{P}_r$ difiere de $\mathbf{P}_c^\top$ en 72 posiciones (el 0,2 %), y el llenado es idéntico al que se obtiene forzando la diagonal con $u = 0$: 5 386 440 coeficientes en ambos casos. Esos intercambios no alteran la estructura que predijo el mínimo grado.

7.8. `spsolve` frente a `splu`

- `spsolve(A, b, permc_spec=...)` —la que usa EduFEM— factoriza y resuelve en una sola llamada (`dgssv`) y descarta $\mathbf{L}$ y $\mathbf{U}$. Es lo natural con un solo vector de cargas.
- `splu(A, permc_spec=..., diag_pivot_thresh=...)` factoriza (`dgstrf`) y devuelve un objeto con `L`, `U`, `perm_r`, `perm_c` y un método `solve(b)` que reutiliza los factores. Conviene cuando hay varios vectores de carga.

Las dos usan la misma biblioteca y el mismo parámetro `permc_spec` con el mismo significado; la nota al pie de la tesis que menciona la opción `MMD_AT_PLUS_A` de `splu` describe, por eso, lo que hace `spsolve`. Un detalle más: `spsolve` usaría UMFPACK en lugar de SuperLU si el paquete `scikits.umfpack` estuviera instalado; en el entorno de EduFEM no lo está, así que el camino es siempre SuperLU.

8. El solucionador de EduFEM, pieza por pieza

8.1. El recorrido completo

La figura 9 resume lo que ocurre al pulsar «Resolver» (F5). La función pública es `solve_system` de `fem/solver.py`, que recibe el proyecto y devuelve $\mathbf{u}$, las reacciones y las matrices intermedias.

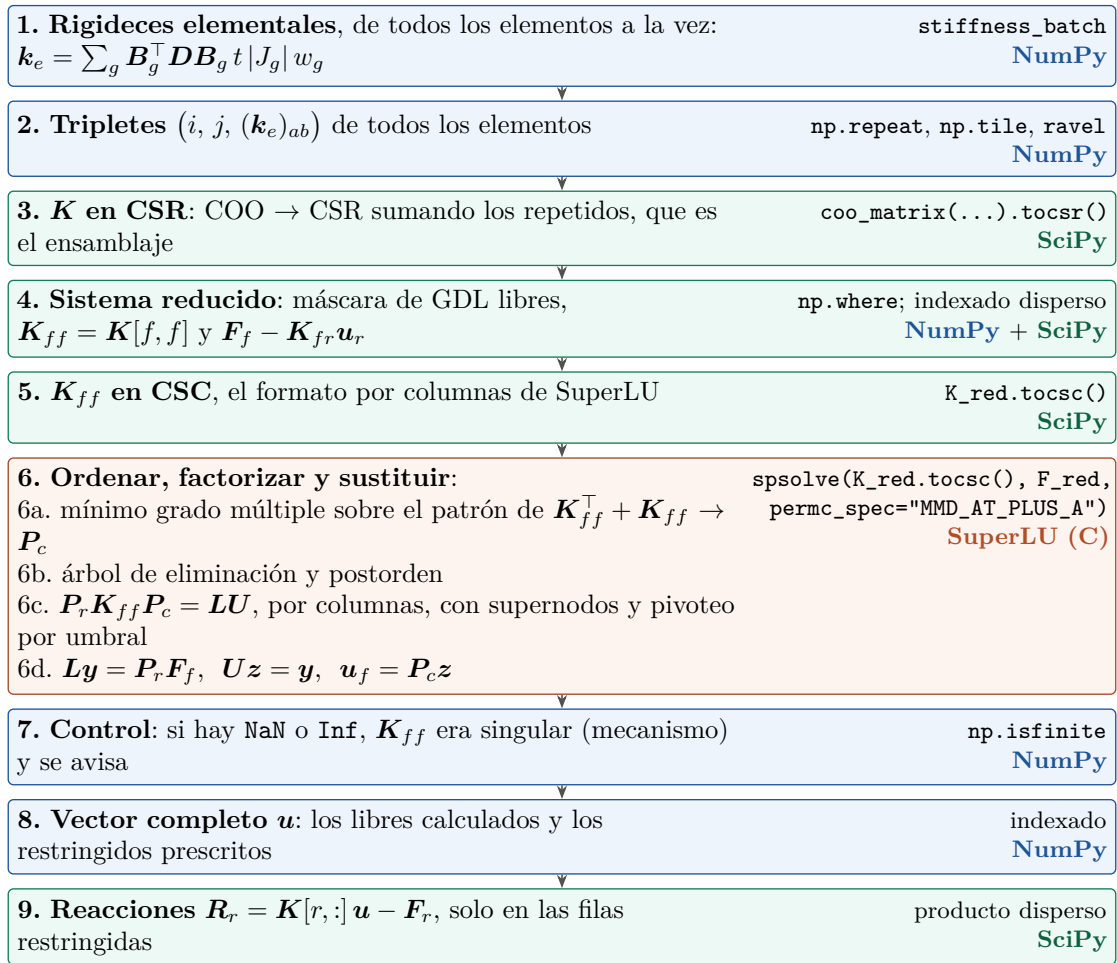


Figura 9: El camino de `solve_system`. Azul: NumPy (arreglos densos); verde agua: `scipy.sparse` (matrices dispersas); naranja: SuperLU, la biblioteca en C que SciPy incluye.

8.2. El código

El ordenamiento se fija en una sola constante, con la medición que lo justifica en el comentario:

Código 2: `config/settings.py`.

```

467 # Ordenamiento de columnas de SuperLU (`permc_spec` de scipy.sparse.linalg.spsolve).
468 #  $K$  es simétrica y definida positiva: "MMD_AT_PLUS_A" (mínimo grado sobre
469 #  $A^T + A$ ) reduce el llenado de la factorización LU frente al default
470 # "COLAMD", pensado para matrices no simétricas. Medido en la membrana de Cook
471 # (spsolve sobre  $K_{red}$  ya en CSC): 2178 GDL 14,3  $\rightarrow$  9,1 ms (1,6x);
472 # 8450 GDL 128  $\rightarrow$  61 ms (2,1x); 33 282 GDL 888  $\rightarrow$  421 ms (2,1x).  $|du| \sim 3e-11$ .

```

```
473 SOLVER_PERMC_SPEC = "MMD_AT_PLUS_A"
```

El sistema reducido se arma con una máscara booleana. $K[\text{free_arr}, :]$ selecciona filas (rápido en CSR) y $[:, \text{free_arr}]$, columnas; el resultado es K_{ff} , ecuación (3):

Código 3: fem/solver.py, apply_boundary_conditions (sin el docstring).

```
44 rest_arr = np.asarray(restrained_dofs, dtype=np.intp)
45 mask = np.ones(n_dof, dtype=bool)
46 if rest_arr.size > 0:
47     mask[rest_arr] = False
48 free_arr = np.where(mask)[0]
49 free_dofs = free_arr.tolist() # API legacy retorna list
50
51 # Indexado por filas luego columnas -- eficiente en CSR y correcto en denso.
52 K_red = K[free_arr, :][:, free_arr]
53 # spsolve requiere CSR o CSC; .tocsr() es no-op si ya es CSR.
54 try:
55     K_red = K_red.tocsr()
56 except AttributeError:
57     pass # K denso (tests de backward-compat): se pasa directo a spsolve
58
59 F_red = F[free_arr].copy()
60
61 if u_prescribed is not None and rest_arr.size > 0:
62     u_r = u_prescribed[rest_arr]
63     if np.any(u_r != 0.0):
64         K_fr = K[free_arr, :][:, rest_arr]
65         F_red -= np.asarray(K_fr @ u_r).ravel()
```

Y la solución propiamente dicha es una línea: todo lo de las secciones 3 a 7 ocurre dentro de spsolve.

Código 4: fem/solver.py, _solve_reduced (sin el docstring).

```
80 from config.settings import SOLVER_PERMC_SPEC
81
82 if not hasattr(K_red, "tocsc"):
83     return spsolve(K_red, F_red)
84
85 return spsolve(K_red.tocsc(), F_red, permc_spec=SOLVER_PERMC_SPEC)
```

La rama con hasattr solo existe para pruebas antiguas que pasan una matriz densa. Después, solve_system controla el resultado, completa u y calcula las reacciones:

Código 5: fem/solver.py, solve_system, pasos 5 a 7.

```
133 u_free = _solve_reduced(K_red, F_red)
134 if np.any(~np.isfinite(u_free)):
135     raise ValueError(
136         "El solver produjo valores NaN o Inf. Verifica que K no sea "
137         "singular: modelo mal restringido, elemento degenerado o E/ν fuera "
138         "de rango pueden causar este problema."
139     )
140
141 # 6. Reconstruir vector completo de desplazamientos.
142 # En DOFs libres: valor calculado. En DOFs restringidos: valor prescrito.
143 # Fancy indexing vectorizado (no loop Python).
144 u = np.zeros(project.total_dof)
145 free_arr = np.asarray(free_dofs, dtype=np.intp)
146 u[free_arr] = u_free
147 if u_prescribed is not None and len(restrained_dofs) > 0:
148     rest_arr = np.asarray(restrained_dofs, dtype=np.intp)
149     u[rest_arr] = u_prescribed[rest_arr]
```

```

150
151 # 7. Calcular reacciones:  $R = K \cdot u - F$ , SOLO en los GDL restringidos.
152 # En los GDL libres  $R$  es  $\sim 0$  (residuo numerico  $\sim 1e-12$ ); ponerlo en 0
153 # exacto evita la multiplicacion densa  $K @ u$  completa (solo se evaluan
154 # las filas restringidas) y deja  $R$  limpio para lectura/reporte.
155 reactions = np.zeros(project.total_dof)
156 if len(restrained_dofs) > 0:
157     rest = np.asarray(restrained_dofs, dtype=np.intp)
158     reactions[rest] = np.asarray(K[rest, :] @ u).ravel() - F[rest]

```

La última línea es la ecuación (4) escrita con la matriz completa: $\mathbf{K}[r, :] \mathbf{u} - \mathbf{F}_r = \mathbf{K}_{rf} \mathbf{u}_f + \mathbf{K}_{rr} \mathbf{u}_r - \mathbf{F}_r$.

8.3. Qué hace NumPy y qué hace SciPy

Biblioteca	Qué hace en el solucionador	Funciones
NumPy	Arreglos densos: el lote de $\mathbf{k}_e$, los tripletes, la máscara de GDL libres, los vectores $\mathbf{F}$, $\mathbf{u}$ y $\mathbf{R}$, el control de NaN.	<code>einsum</code> , <code>matmul</code> , <code>repeat</code> , <code>tile</code> , <code>where</code> , <code>isfinite</code>
<code>scipy.sparse</code>	Matrices dispersas: COO, CSR y CSC, la suma de repetidos, la extracción de $\mathbf{K}_{ff}$ y $\mathbf{K}_{fr}$, los productos matriz–vector.	<code>coo_matrix</code> , <code>tocsr</code> , <code>tocsc</code> , <code>indexado</code> , <code>@</code>
SuperLU, dentro de <code>scipy.sparse.linalg</code>	Ordenamiento, árbol de eliminación, factorización con supernodos y pivoteo, sustituciones.	<code>spsolve</code>

NumPy [22] tiene su propio solucionador, `numpy.linalg.solve`, pero es denso: también factoriza —llama a la rutina `gesv` de LAPACK, una LU con pivoteo parcial—, pero sobre los n^2 coeficientes. Es la columna «LU densa» de la tabla 2. `numpy.linalg.inv`, por su parte, factoriza y después resuelve n sistemas, uno por cada columna de la identidad.

8.4. Traza de una resolución real

Todo lo anterior, con los números de una corrida del motor sobre la membrana de Cook de 8×8 elementos Q9:

Membrana de Cook, 8×8 Q9

1. **Malla:** 289 nodos, 64 elementos, 578 GDL.
2. **Lote:** 64 matrices $\mathbf{k}_e$ de $18 \times 18 \rightarrow 20\,736$ tripletes.
3. `coo_matrix(...).tocsr()`: suma 3836 repetidos $\rightarrow \mathbf{K}$ con 16 900 no nulos, el 5,06 % de 578^2 .
4. **Restricciones:** 34 GDL restringidos (los nodos del borde empotrado) $\rightarrow n = 544$ libres; $\mathbf{K}_{ff}$ con 15 600 no nulos.
5. `spsolve(K_red.tocsc(), F_red, permc_spec="MMD_AT_PLUS_A")`: $\mathbf{L} + \mathbf{U}$ con 27 104 coeficientes, 1,7 veces `nnz(Kff)` (sin reordenar habrían sido 232 880); tiempo: 2,0 ms.
6. **Control:** residuo $\|\mathbf{K}_{ff} \mathbf{u}_f - \mathbf{F}_f\| / \|\mathbf{F}_f\| = 1,1 \times 10^{-12}$. Desplazamiento vertical en el centro del lado cargado, (48, 52): $u_y = 23,93$, el mismo valor que reporta la verificación de la tesis para esta malla (docs/vyv/datos/cook.csv; valor de referencia publicado, 23,96).
7. **Reacciones:** $\sum R_y = -1,0000$ frente a $\sum F_y = 1,0000$; $|\sum R_x| = 5,8 \times 10^{-14}$. El equilibrio global se cumple.

8.5. Cómo se comprueba que el solucionador está bien

- **Residuo:** $\|\mathbf{K}_{ff} \mathbf{u}_f - \mathbf{F}_f\| / \|\mathbf{F}_f\|$ del orden de 10^{-12} en todas las mallas (tabla 2).
- **Equilibrio:** la suma de reacciones equilibra la suma de cargas, como en la traza.

- **Regresión:** `tests/test_solver_regression.py` compara el motor por lotes con la versión legible elemento a elemento (error relativo $\leq 10^{-9}$), y `tests/test_fem.py` verifica Q4, Q9 y las cargas superficiales.
- **Ordenamiento:** `tests/bench_timing.py` mide COLAMD frente a MMD_AT_PLUS_A sobre el mismo sistema; cambiar el ordenamiento cambia $\mathbf{u}$ solo en el redondeo (del orden de 10^{-11} , según el comentario de `config/settings.py`).
- **Verificación y validación:** la membrana de Cook, la viga de Timoshenko y las soluciones manufacturadas de la tesis ejercitan el mismo camino.

9. Preguntas que pueden aparecer en la defensa

¿EduFEM invierte la matriz de rigidez?

No. Factoriza $\mathbf{K}_{ff} = \mathbf{LU}$ con SuperLU y obtiene $\mathbf{u}_f$ por sustitución hacia adelante y hacia atrás. La inversa sería llena (medido: el 100 % de coeficientes no nulos), costaría tres veces más operaciones incluso en denso y con 33 024 GDL ocuparía 8,7 GB.

Entonces, ¿qué significa $\mathbf{u} = \mathbf{K}^{-1}\mathbf{F}$ en la tesis?

Es la notación de la solución única, que existe porque $\mathbf{K}_{ff}$ es SPD. El algoritmo que la calcula es factorizar y sustituir.

¿Qué es la factorización LU?

La eliminación de Gauss con los multiplicadores guardados: $\mathbf{A} = \mathbf{LU}$, donde $\mathbf{L}$ (triangular inferior) reúne los multiplicadores y $\mathbf{U}$ (triangular superior) es lo que queda al eliminar. Después, $\mathbf{Ly} = \mathbf{F}$ y $\mathbf{Uu} = \mathbf{y}$.

Si $\mathbf{K}$ es SPD, ¿por qué no Cholesky?

Cholesky haría la mitad de operaciones y de memoria, pero SciPy no la trae en formato disperso: habría que agregar `scikit-sparse` (CHOLMOD), una dependencia binaria más para el instalador. Con MMD_AT_PLUS_A y pivotes diagonales, la LU tiene la misma estructura que Cholesky ($\mathbf{U} = \mathbf{DL}^\top$): la diferencia es un factor cercano a 2, no un cambio de orden. Queda como recomendación de la tesis.

¿Qué es SuperLU?

Una biblioteca en C de factorización LU dispersa con pivoteo parcial (Demmel, Gilbert, Li y otros), incluida en SciPy. «Super» por los supernodos: grupos de columnas con la misma estructura que se procesan como bloques densos, lo que la hace rápida.

¿Qué es el llenado y por qué importa?

Son los coeficientes nulos en $\mathbf{K}$ que dejan de serlo en $\mathbf{L}$ o $\mathbf{U}$. Cuestan memoria y tiempo, y dependen del orden en que se eliminan las incógnitas. Con 8320 GDL: sin reordenar, 52 485 872 coeficientes y 150 s; con mínimo grado, 944 056 y 0,13 s.

¿Qué hace el ordenamiento de mínimo grado?

Elige, en cada paso, eliminar la incógnita con menos conexiones, porque eliminar una incógnita conecta entre sí a todos sus vecinos: la de menor grado es la que menos llenado crea en ese paso. Es una heurística; encontrar el orden óptimo es un problema NP-completo.

¿Por qué $\mathbf{K}^\top + \mathbf{K}$ si $\mathbf{K}$ ya es simétrica?

Porque SuperLU está hecho para matrices generales y el mínimo grado necesita un grafo no dirigido: $\mathbf{A}^\top + \mathbf{A}$ simetriza el patrón. Para $\mathbf{K}$, $\mathbf{K}^\top + \mathbf{K} = 2\mathbf{K}$ tiene el mismo patrón, así que es el mínimo grado sobre el grafo de la propia malla.

¿Por qué no el ordenamiento por defecto de SciPy (COLAMD)?

COLAMD ordena pensando en $A^T A$, lo que protege contra cualquier pivoteo en matrices no simétricas. Para K , $A^T A = K^2$ une GDL a dos elementos de distancia, un modelo más grueso: con 33 024 GDL da más llenado (11,0 contra 5,2 veces $\text{nnz}(K_{ff})$), hace 2,6 veces más operaciones y tarda 2,6 veces más.

¿El reordenamiento cambia la solución?

No: resuelve el mismo sistema con las ecuaciones en otro orden, $(PKP^T)(Pu) = PF$. Solo cambia el redondeo, en el orden de 10^{-11} relativo.

¿Hay pivoteo? ¿Es estable?

SuperLU usa pivoteo parcial por umbral ($u = 1$ en `spsolve`). En K_{ff} SPD elige casi siempre la diagonal (difiere en 72 de 33 024 posiciones) sin cambiar el llenado. Una matriz SPD es estable incluso sin pivoteo: sus pivotes son positivos y los coeficientes no crecen.

¿Qué es COO y por qué después CSR y CSC?

COO es la lista de tripletes (fila, columna, valor); admite repetidos, que se suman: eso es el ensamblaje. CSR agrupa por filas, lo que sirve para extraer K_{ff} y calcular reacciones; CSC agrupa por columnas, que es como trabaja SuperLU.

¿Qué hace NumPy y qué hace SciPy?

NumPy maneja los arreglos densos: el lote de k_e , los tripletes, las máscaras y los vectores. SciPy maneja las matrices dispersas y el solucionador; SuperLU es código C compilado dentro de SciPy.

¿Por qué un solucionador directo y no uno iterativo (gradiente conjugado)?

En los tamaños de uso formativo el directo es robusto —no depende del condicionamiento ni de un preconditionador— y rápido: 1,1 s con 33 024 GDL en un portátil de 2012. El iterativo preconditionado queda entre las recomendaciones para mallas muy grandes.

¿Qué pasa si el modelo está mal restringido?

K_{ff} es singular: aparece un pivote nulo, SuperLU lo informa, SciPy devuelve NaN y EduFEM detiene el cálculo con un mensaje que apunta a un modelo mal restringido o a un elemento degenerado. El validador de salud del modelo avisa antes de llegar a resolver.

¿Cómo sé que la solución es correcta?

Residuo relativo del orden de 10^{-12} , equilibrio de reacciones, el test de regresión contra la versión legible del motor y la verificación y validación de la tesis (Cook, Timoshenko, soluciones manufacturadas).

A. Conteo de operaciones

Se cuenta una suma, una resta, un producto o una división como una operación (*flop*).

Factorización LU. En el paso k hay $n - k$ multiplicadores (una división cada uno) y $(n - k)^2$ actualizaciones $a_{ij} - \ell_{ik}a_{kj}$ (un producto y una resta cada una):

$$\sum_{k=1}^{n-1} [(n - k) + 2(n - k)^2] = \sum_{m=1}^{n-1} (m + 2m^2) \approx \frac{2}{3}n^3.$$

Sustituciones. La fila i de la ecuación (9) hace $i - 1$ productos y $i - 1$ restas: $\sum_i 2(i - 1) \approx n^2$. La sustitución hacia atrás cuesta lo mismo. Total: $2n^2$.

Inversa. Una vez factorizada la matriz, la columna j de $\mathbf{A}^{-1}$ resuelve $\mathbf{LU}\mathbf{x} = \mathbf{e}_j$. En la sustitución hacia adelante, las primeras $j - 1$ componentes de $\mathbf{y}$ son nulas, así que cuesta $\approx (n - j)^2$; la sustitución hacia atrás cuesta n^2 completa. Sumando sobre las n columnas:

$$\frac{2}{3}n^3 + \sum_{j=1}^n (n - j)^2 + n \cdot n^2 \approx \frac{2}{3}n^3 + \frac{1}{3}n^3 + n^3 = 2n^3.$$

Caso disperso. Si la columna j de $\mathbf{L}$ tiene c_j no nulos debajo de la diagonal y la fila j de $\mathbf{U}$ tiene r_j a la derecha de la diagonal, el paso j hace c_j divisiones y $c_j r_j$ actualizaciones:

$$\text{operaciones} = \sum_{j=1}^n (c_j + 2 c_j r_j),$$

que con patrón simétrico ($r_j = c_j$) es $\sum_j (c_j + 2c_j^2)$. Es la cuenta de la tabla 4, hecha sobre los factores que devuelve SuperLU. El llenado entra *al cuadrado*: por eso reducirlo tiene un efecto tan grande sobre el tiempo. En el caso denso, $c_j = r_j = n - j$ y se recupera el $\frac{2}{3}n^3$ de arriba.

B. Cómo reproducir las cifras

Desde la raíz del repositorio, con el entorno del proyecto:

```
python docs/teoria/solucionador/generar_datos.py          # todo (unos 5 minutos)
python docs/teoria/solucionador/generar_datos.py --rapido # sin los casos de mas de 10 s
cd docs/teoria/solucionador
pdflatex solucionador_disperso.tex                       # dos veces, por las referencias
```

El guion escribe `datos/cifras.tex` (las cifras del texto), las filas de tabla `datos/tabla_*.tex`, las mediciones en bruto `datos/mediciones.json` y las figuras de `figuras/`. Antes de medir, verifica en aritmética exacta los ejemplos 1 a 4; si alguno dejara de cumplirse, se detiene sin escribir nada.

Referencias

- [1] Golub GH, Van Loan CF. Matrix computations. 4.^a ed. Baltimore: Johns Hopkins University Press; 2013.
- [2] Trefethen LN, Bau D. Numerical linear algebra. Philadelphia: SIAM; 1997.
- [3] Higham NJ. Accuracy and stability of numerical algorithms. 2.^a ed. Philadelphia: SIAM; 2002.
- [4] Davis TA. Direct methods for sparse linear systems. Philadelphia: SIAM; 2006.
- [5] Davis TA, Rajamanickam S, Sid-Lakhdar WM. A survey of direct methods for sparse linear systems. Acta Numer. 2016;25:383-566.
- [6] George A, Liu JWH. Computer solution of large sparse positive definite systems. Englewood Cliffs: Prentice-Hall; 1981.
- [7] Parter S. The use of linear graphs in Gauss elimination. SIAM Rev. 1961;3(2):119-30.
- [8] Rose DJ. A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations. En: Read RC, editor. Graph theory and computing. New York: Academic Press; 1972. p. 183-217.
- [9] Markowitz HM. The elimination form of the inverse and its application to linear programming. Manage Sci. 1957;3(3):255-69.
- [10] Tinney WF, Walker JW. Direct solutions of sparse network equations by optimally ordered triangular factorization. Proc IEEE. 1967;55(11):1801-9.
- [11] George A. Nested dissection of a regular finite element mesh. SIAM J Numer Anal. 1973;10(2):345-63.

- [12] Yannakakis M. Computing the minimum fill-in is NP-complete. *SIAM J Algebraic Discrete Methods*. 1981;2(1):77-9.
- [13] Liu JWH. Modification of the minimum-degree algorithm by multiple elimination. *ACM Trans Math Softw*. 1985;11(2):141-53.
- [14] George A, Liu JWH. The evolution of the minimum degree ordering algorithm. *SIAM Rev*. 1989;31(1):1-19.
- [15] George A, Ng E. Symbolic factorization for sparse Gaussian elimination with partial pivoting. *SIAM J Sci Stat Comput*. 1987;8(6):877-98.
- [16] Davis TA, Gilbert JR, Larimore SI, Ng EG. A column approximate minimum degree ordering algorithm. *ACM Trans Math Softw*. 2004;30(3):353-76.
- [17] Liu JWH. The role of elimination trees in sparse factorization. *SIAM J Matrix Anal Appl*. 1990;11(1):134-72.
- [18] Gilbert JR, Peierls T. Sparse partial pivoting in time proportional to arithmetic operations. *SIAM J Sci Stat Comput*. 1988;9(5):862-74.
- [19] Demmel JW, Eisenstat SC, Gilbert JR, Li XS, Liu JWH. A supernodal approach to sparse partial pivoting. *SIAM J Matrix Anal Appl*. 1999;20(3):720-55.
- [20] Li XS. An overview of SuperLU: algorithms, implementation, and user interface. *ACM Trans Math Softw*. 2005;31(3):302-25.
- [21] Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat Methods*. 2020;17(3):261-72.
- [22] Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, et al. Array programming with NumPy. *Nature*. 2020;585(7825):357-62.
- [23] Bathe KJ. Finite element procedures. 2.^a ed. Watertown (MA): K.J. Bathe; 2014.